# BRIEFINGS

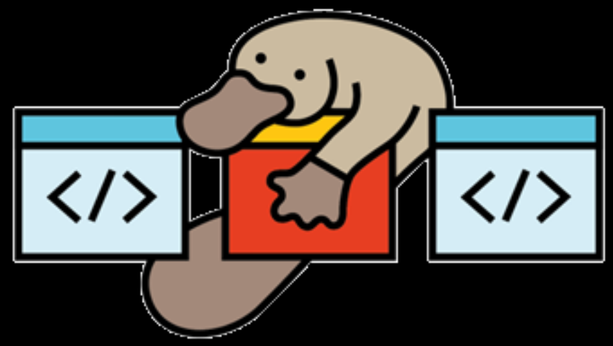
black hat

black hat
USA
2026

# PLaTypus: Killing Code-Reuse at the Module Boundary

Speaker: Apostolos Chatzianagnostou

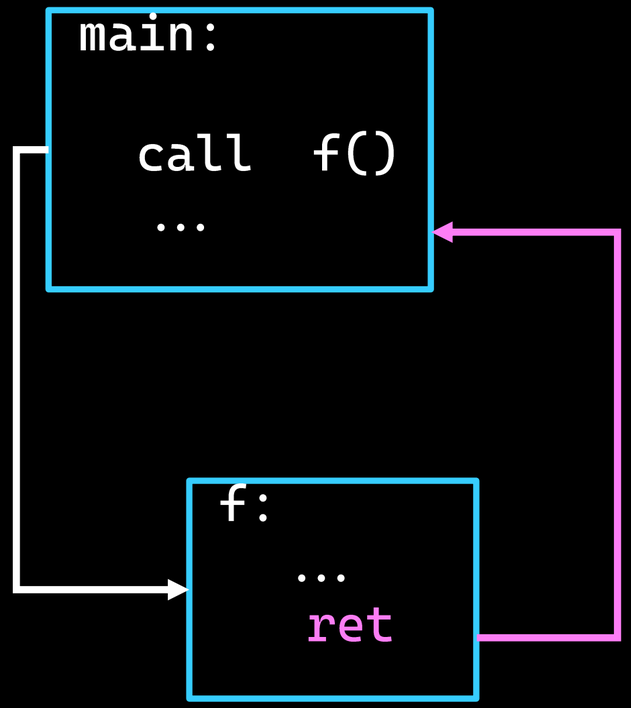
Collaborators: Marcos Bajo, Christian Rossow



# Memory War: Chronicles

⚔️ Ret2libc (1997)

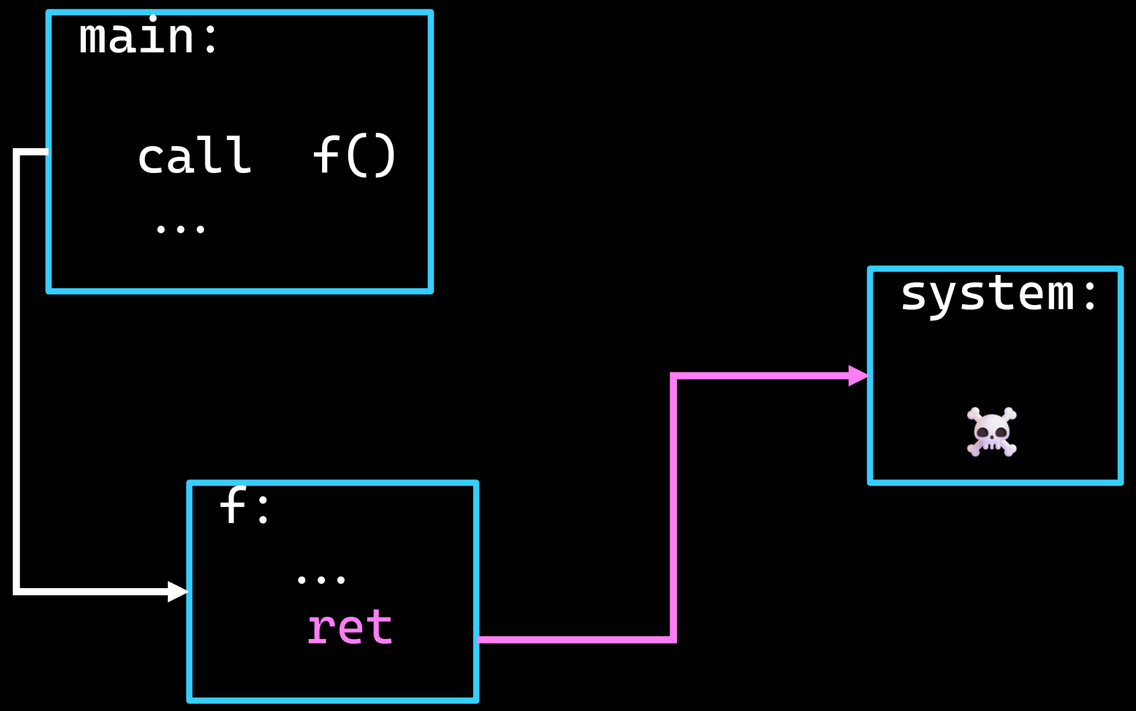
- *And so it begins...*



# Memory War: Chronicles

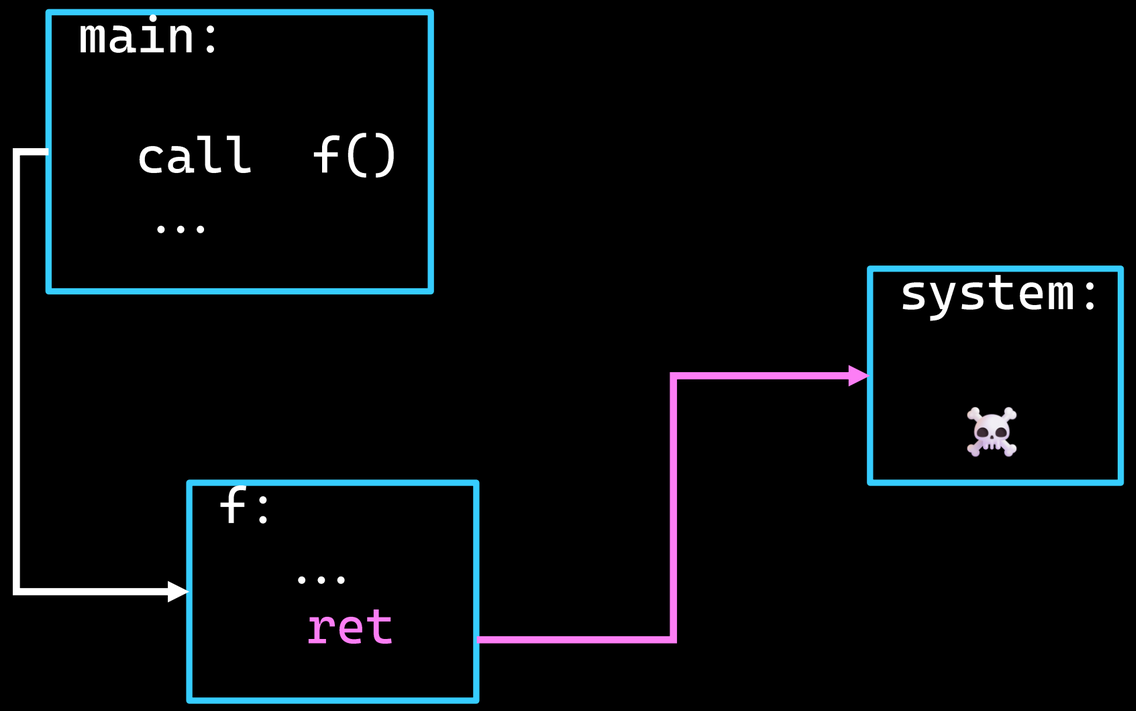
⚔️ Ret2libc (1997)

- *And so it begins...*



# Memory War: Chronicles

- ⚔️ Ret2libc (1997)
  - *And so it begins...*
- ⚔️ ROP (2007)
- ⚔️ JOP (2011)
- ⚔️ SROP (2014)



# Memory War: Chronicles

- Stack canaries (~2000)
- ASLR (2003)
- DEP/NX (2004)

# Memory War: Chronicles

- Stack canaries (~2000)
- ASLR (2003)
- DEP/NX (2004)
- Control Flow Integrity (2005)
  - Control Flow Guard (2014)
  - LLVM CFI (2015)
  - Intel CET (2020)

Powerful in theory, hard to enforce…

# Memory War: Chronicles

- ⚔ COOP (2015)
- ⚔ FOP (2018)
- ⚔ CFOP (2024)
- ⚔ SFOP (2026)

# Memory War: Chronicles

- ⚔️ COOP (2015)
- ⚔️ FOP (2018)
- ⚔️ CFOP (2024)
- ⚔️ SFOP (2026)

Code-reuse attacks still an issue…

# Memory War: Chronicles

- ⚔ COOP (2015)
- ⚔ FOP (2018)
- ⚔ CFOP (2024)
- ⚔ SFOP (2026)

Code-reuse attacks still an issue…

But they all share one thing:

Cross-module transitions!

# Who We Are

CISPA
HELMHOLTZ CENTER FOR INFORMATION SECURITY


**Apostolos Chatzianagnostou**
apostolos.chatzianagnostou@cispa.de


- PhD student at CISPA


**Marcos Bajo**
aka ***h3xduck***
h3xduck@gmail.com


- PhD student at CISPA
- *https://h3xduck.github.io*


**Christian Rossow**
rossow@cispa.de


- Faculty at CISPA
- CS Professor at TU Dortmund
- Leader of the *Systems Security Group*

We build, break, and explore. Reach out!

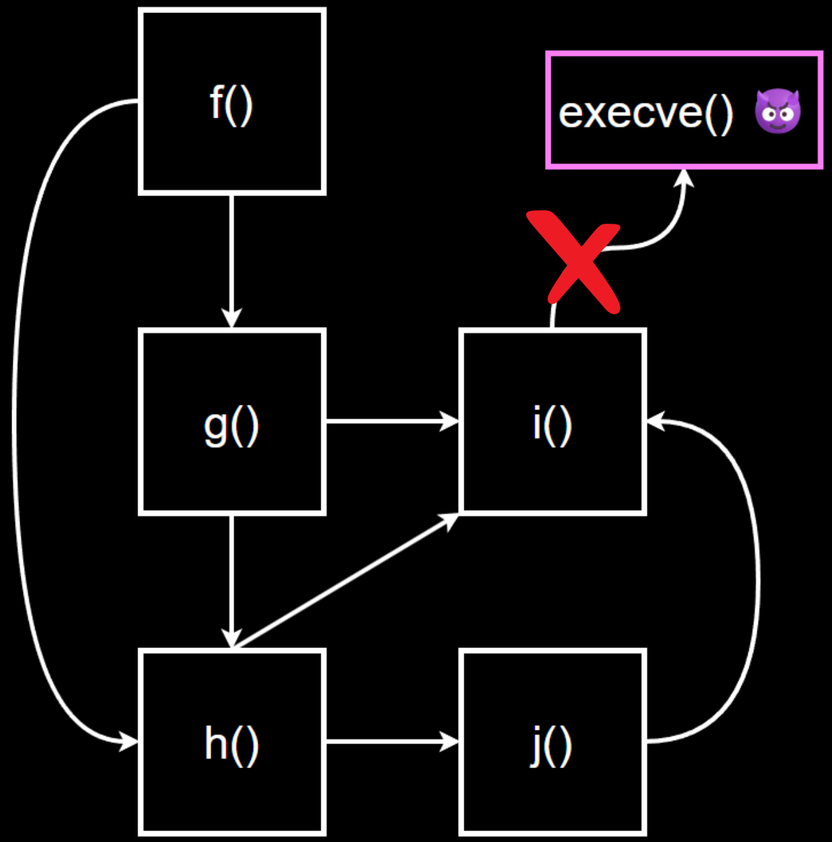
# Agenda

1. Userspace CFI and debloating approaches
2. Motivation and goals of PLaTypus
3. Design, methodology and challenges
4. Evaluation and discussion

# BACKGROUND

# Control Flow Integrity (CFI)

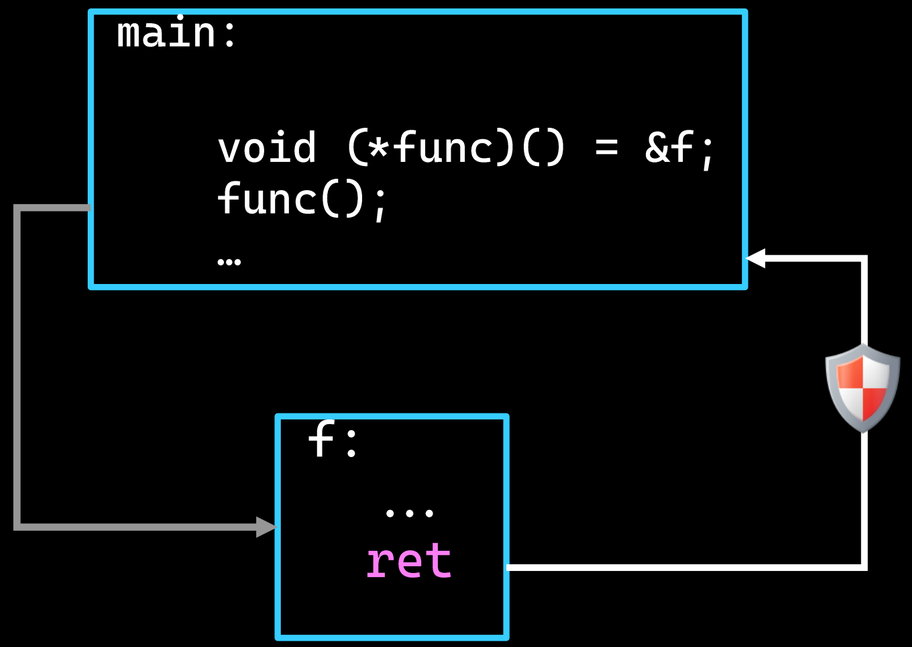
- Creation (statically) of Control Flow Graph (CFG)
  Which indirect transitions are expected/benign?
- Enforcement of CFG through code instrumentation
- Code-reuse: solved
  Well…



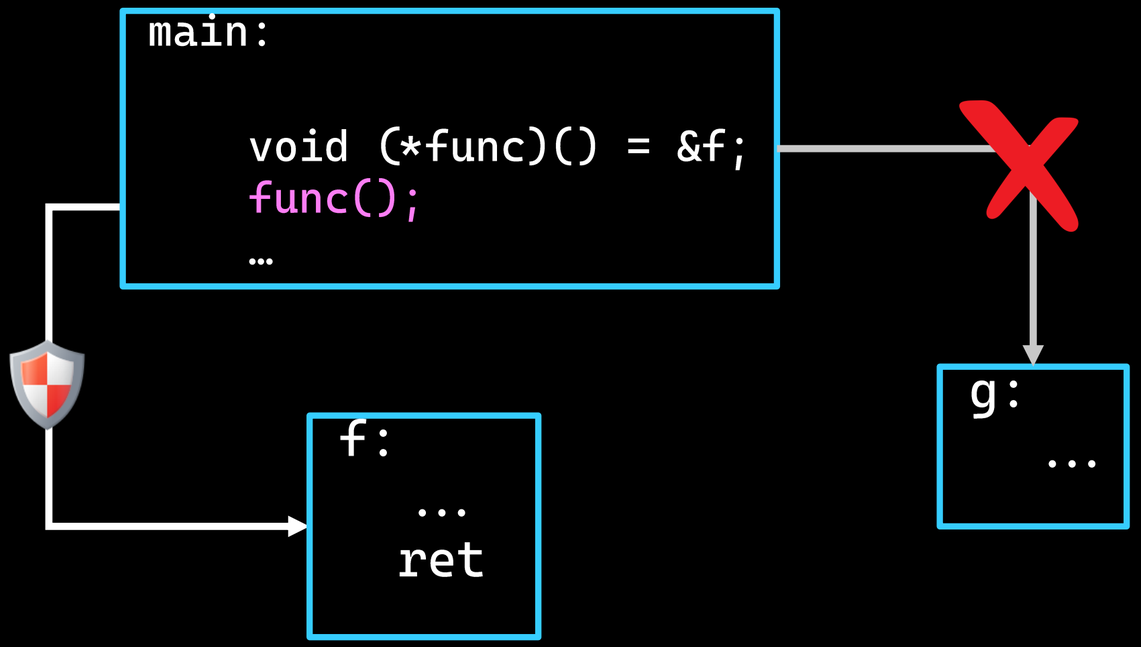
# CFI Taxonomy

Based on the protected edges:

1. Backward-edge CFI

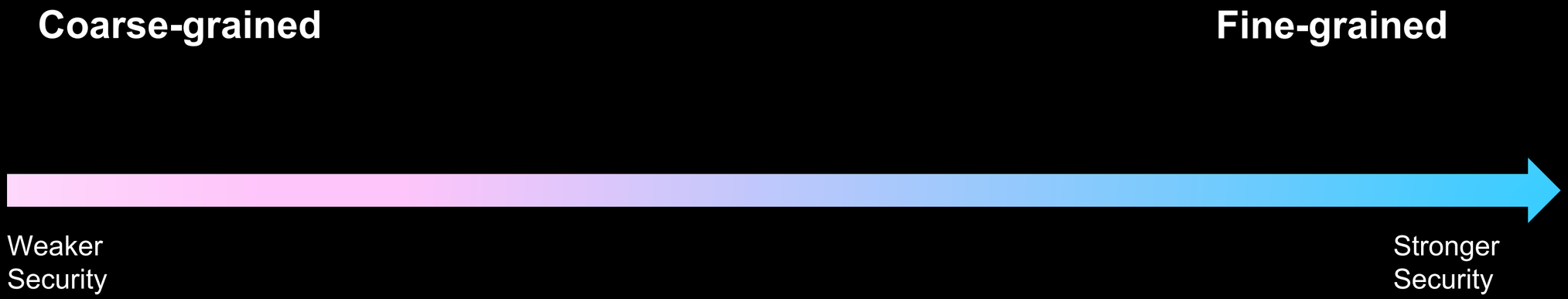



2. Forward-edge CFI



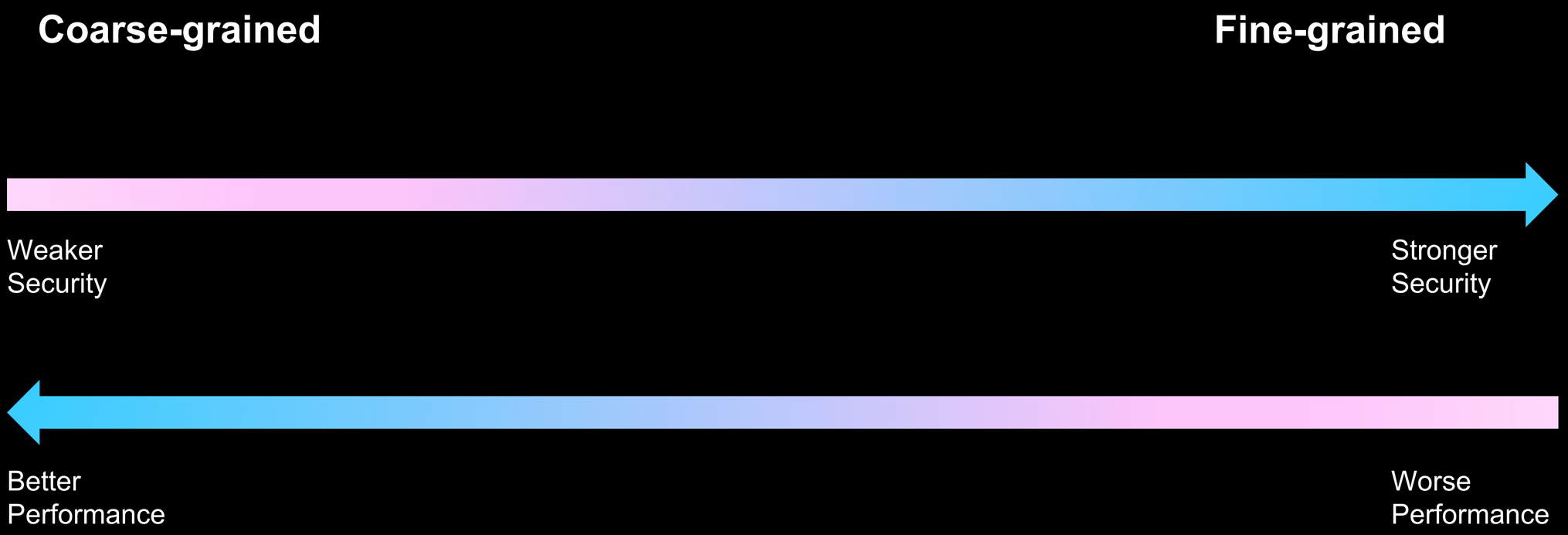
# CFI Taxonomy

Based on security:



# CFI Taxonomy

Based on security:



# CFI Limitations

- Accurate CFG construction is hard

  Static analysis limitations ⟶ Overapproximation

- Incomplete, leaving pointers unprotected

  Example: C++ coroutines

- Protected/unprotected interop is hard

  Needed for third-party libraries/modules

Fine-grained CFI: limited deployment

# CFI Limitations

- Accurate CFG construction is hard

  Static analysis limitations ⟶ Overapproximation

- Incomplete, leaving pointers unprotected

  Example: C++ coroutines

- Protected/unprotected interop is hard

  Needed for third-party libraries/modules

Fine-grained CFI: limited deployment

Deployed schemes in practice:

- Intel CET (coarse-grained)
- ARM BTI (coarse-grained)
- Control Flow Guard (coarse-grained)
- LLVM CFI (fine(r)-grained)

# Intel CET

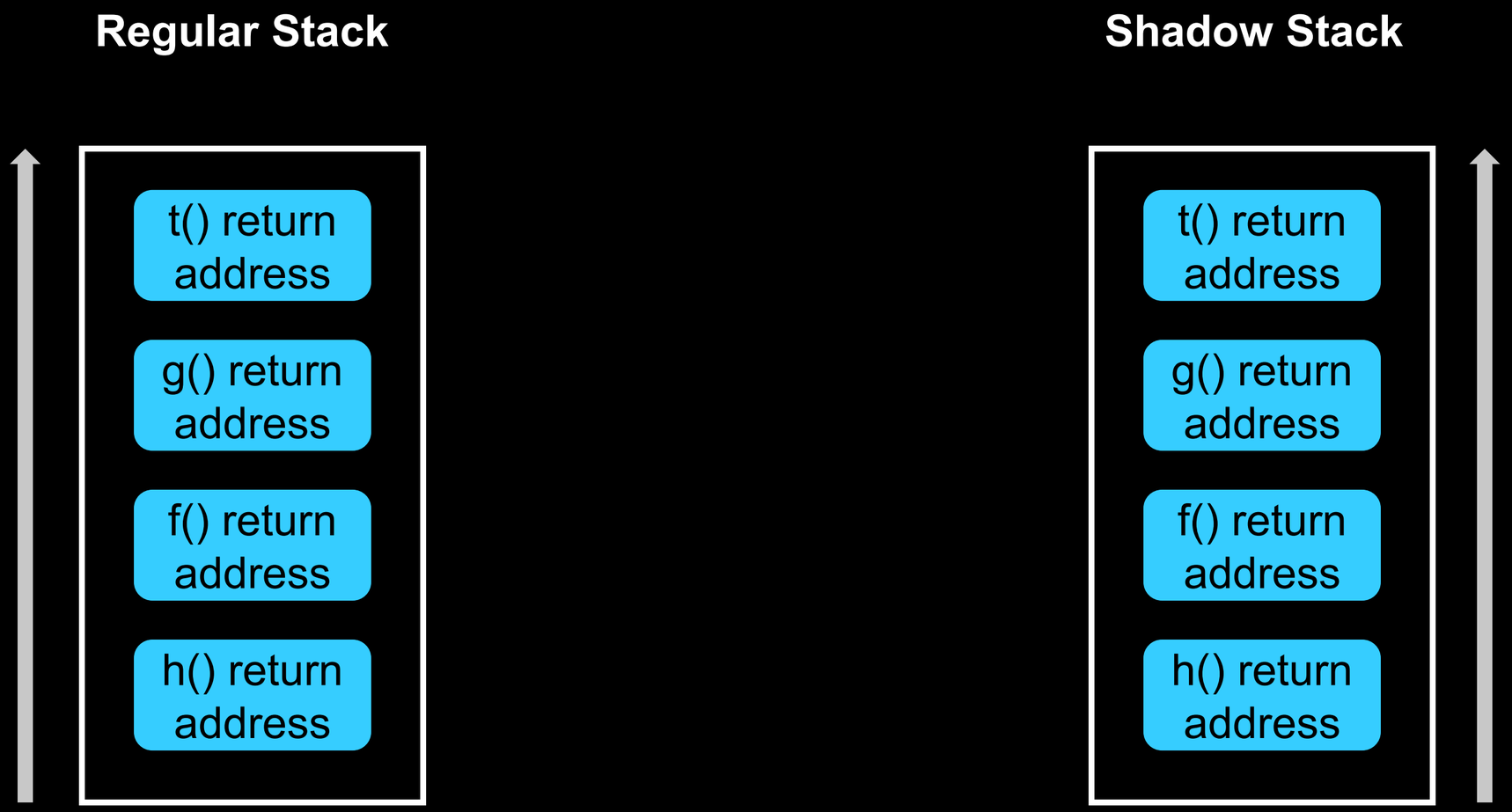
- Hardware-enforced CFI

- Two main components:
  1. Shadow Stack for backward edges
  2. Indirect Branch Tracking (IBT) for forward edges

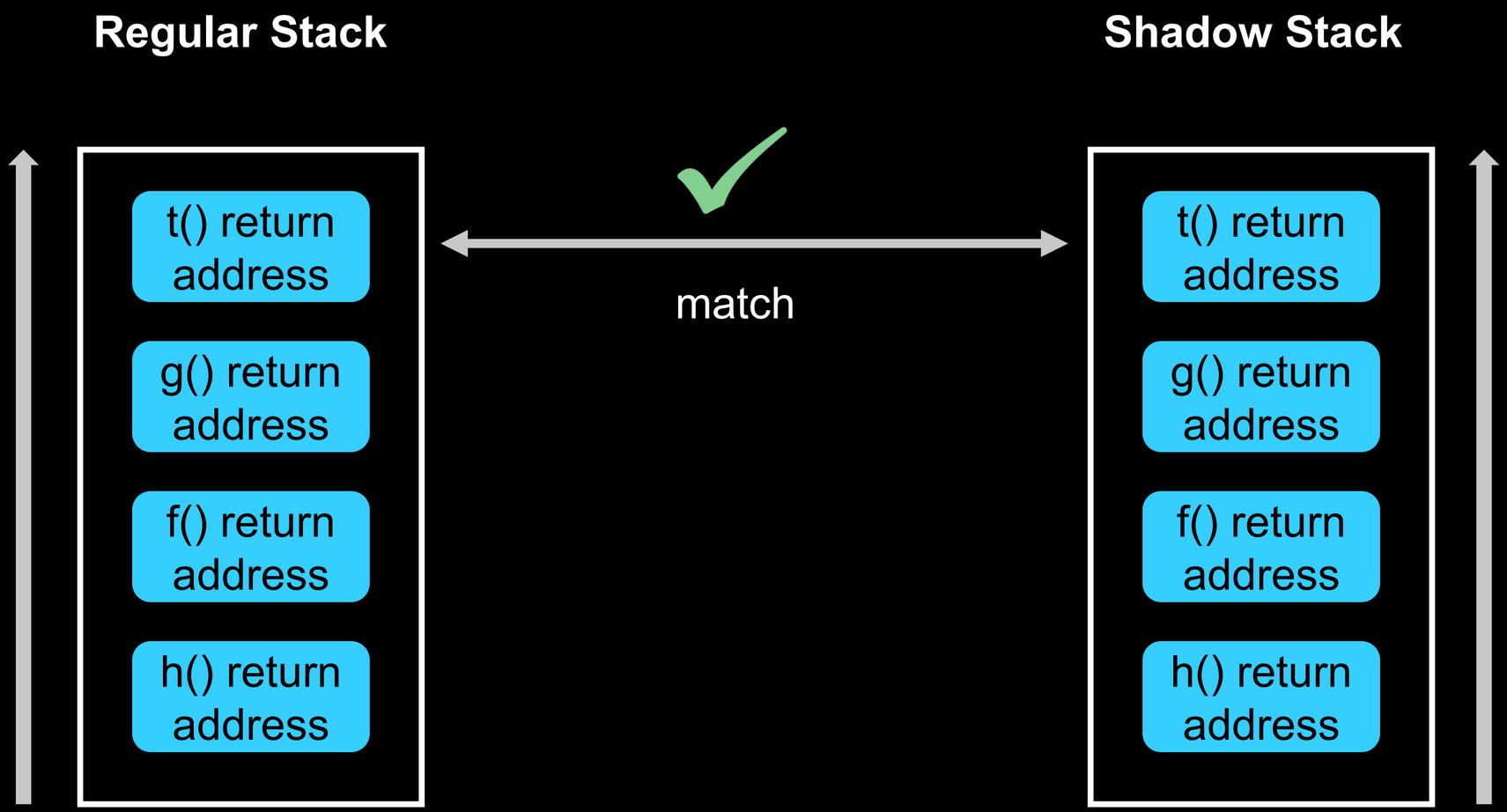
- Supported on Intel 11th Gen and later

- Available on both Windows and Linux

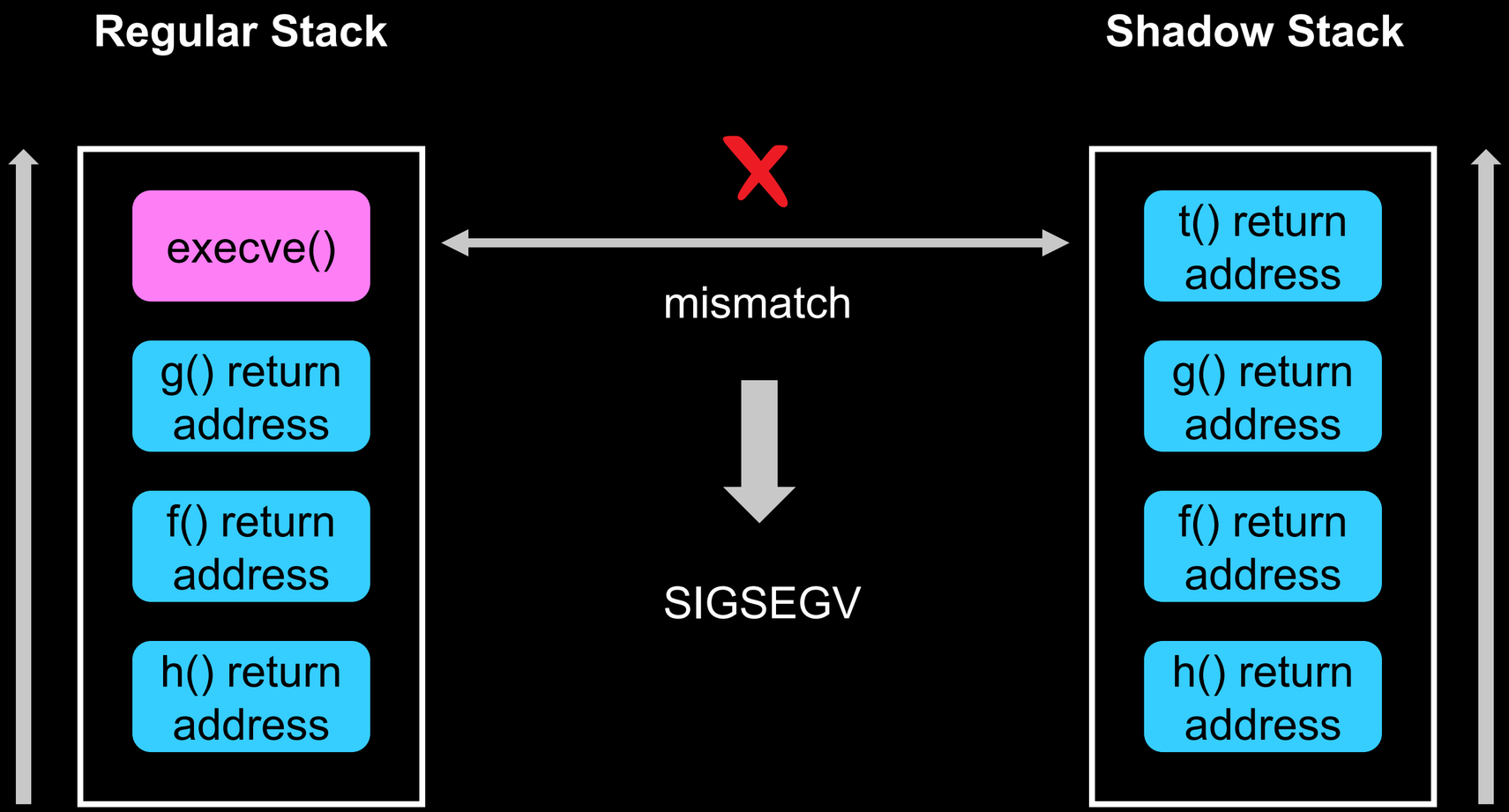
# Intel CET – Shadow Stacks



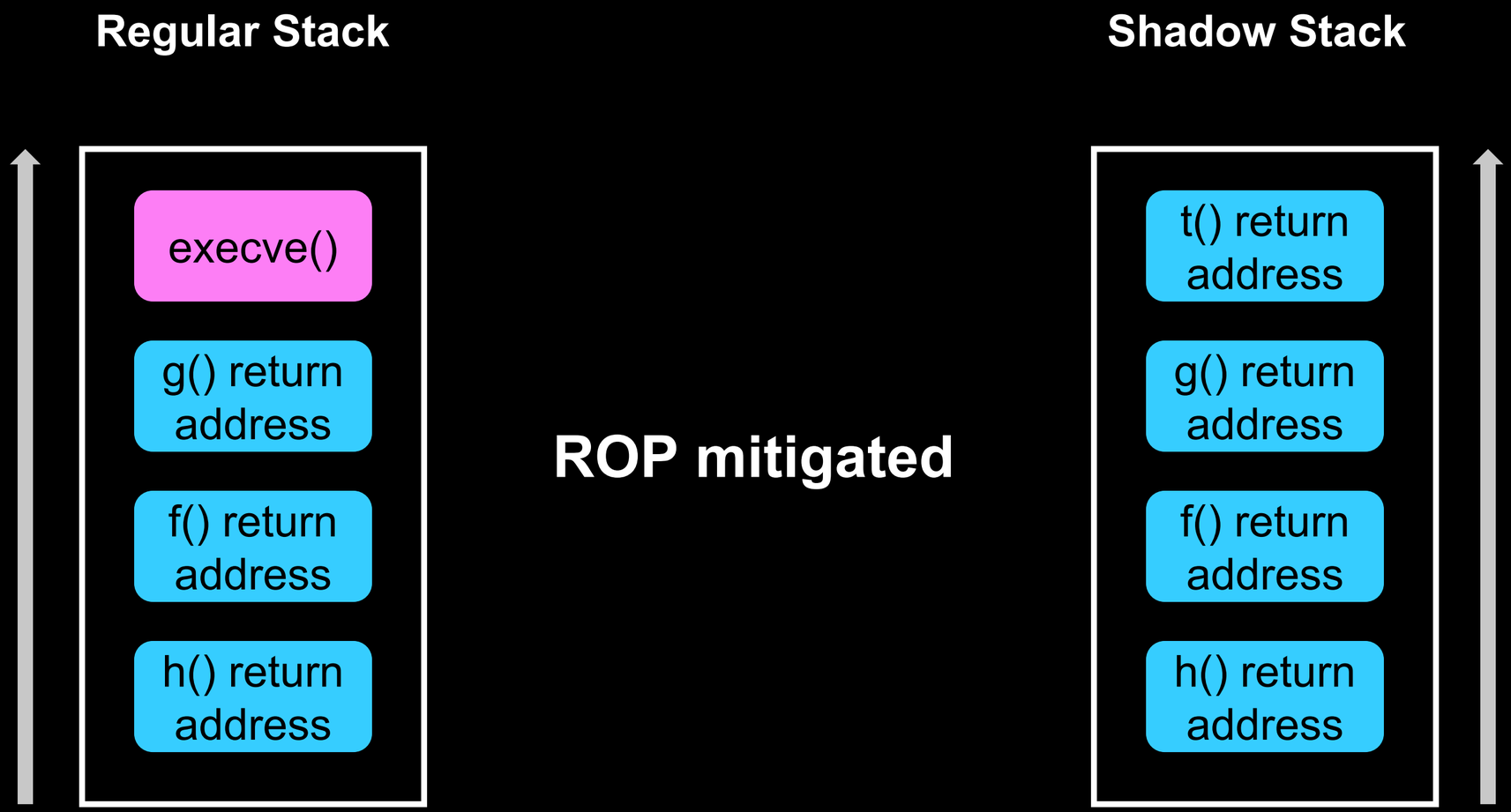
# Intel CET – Shadow Stacks



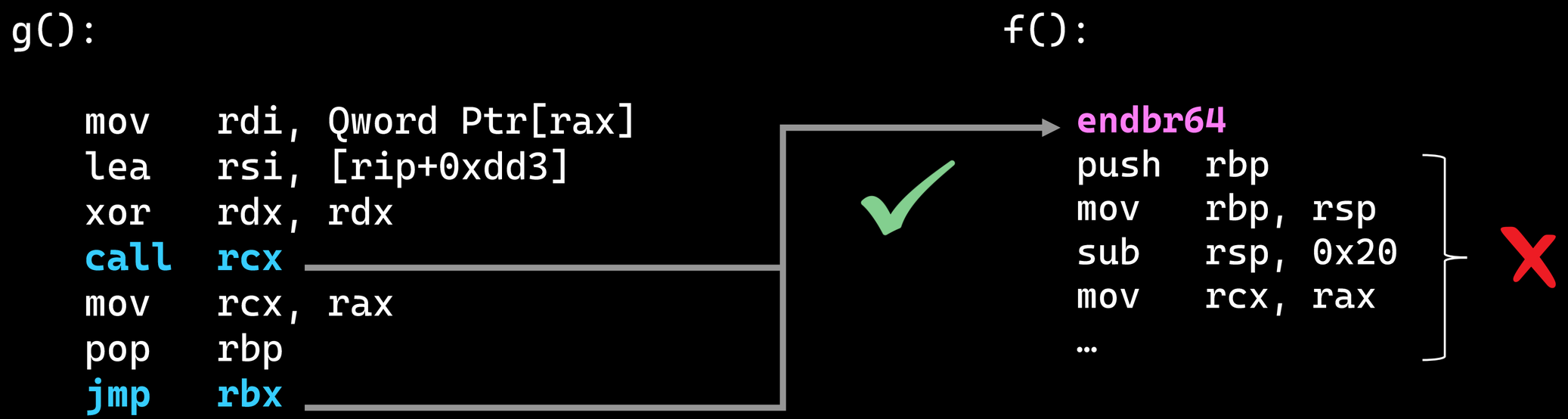
# Intel CET – Shadow Stacks



# Intel CET – Shadow Stacks



# Intel CET – IBT



# ARM BTI

- Protects forward edges
- Similar to Intel’s IBT but in ARM hardware
- Can differentiate between *call targets* and *jump targets*
- Available on Apple, Android, Linux and Windows systems

# Control Flow Guard

- Protects forward edges
- Microsoft’s implementation of Intel IBT
- Software-enforced
- A bitmap marks valid indirect call/jump targets

# LLVM CFI

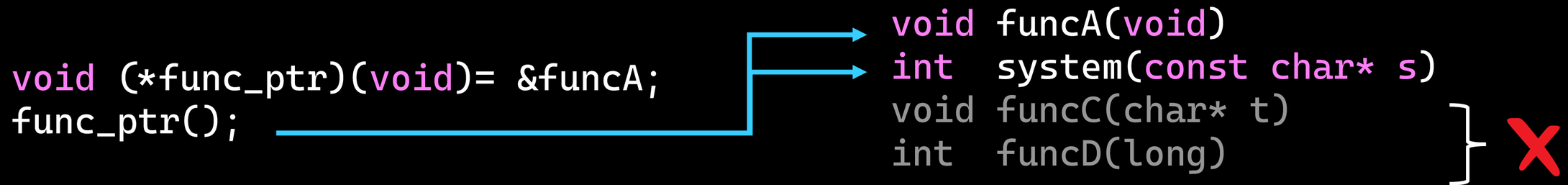
- Type-based CFI
- Software-enforced
- Applicable also to virtual calls in C++

```
void (*func_ptr)(void)= &funcA;
func_ptr();
```

```
void funcA(void)
int  funcB(const char* s)
void funcC(char* t)
int  funcD(long)
```

# LLVM CFI

- Supports modularity (*cross-DSO* mode)
- Transitions to uninstrumented libraries are allowed
- Cannot compile glibc



# Debloating

- Remove code that is not needed

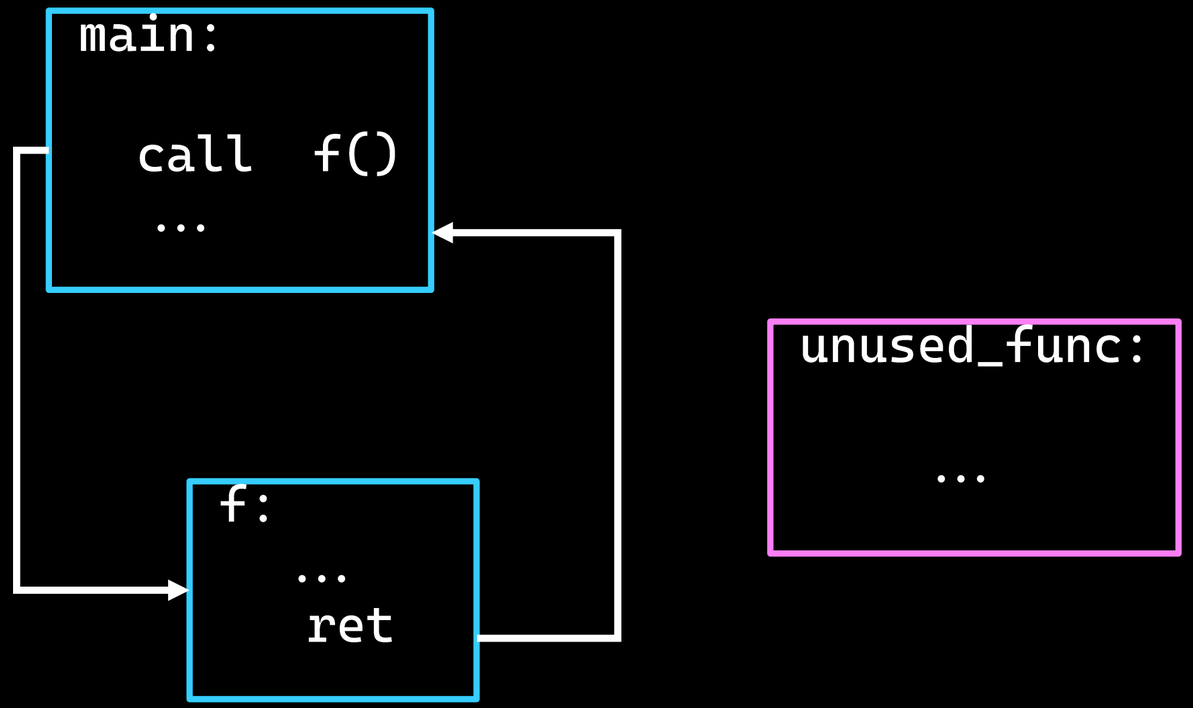
  E.g., dead code

- Available gadgets are reduced

  Attack surface shrinks

- Especially useful in libraries

  Plethora of gadgets and unused code



# Debloating

- Remove code that is not needed

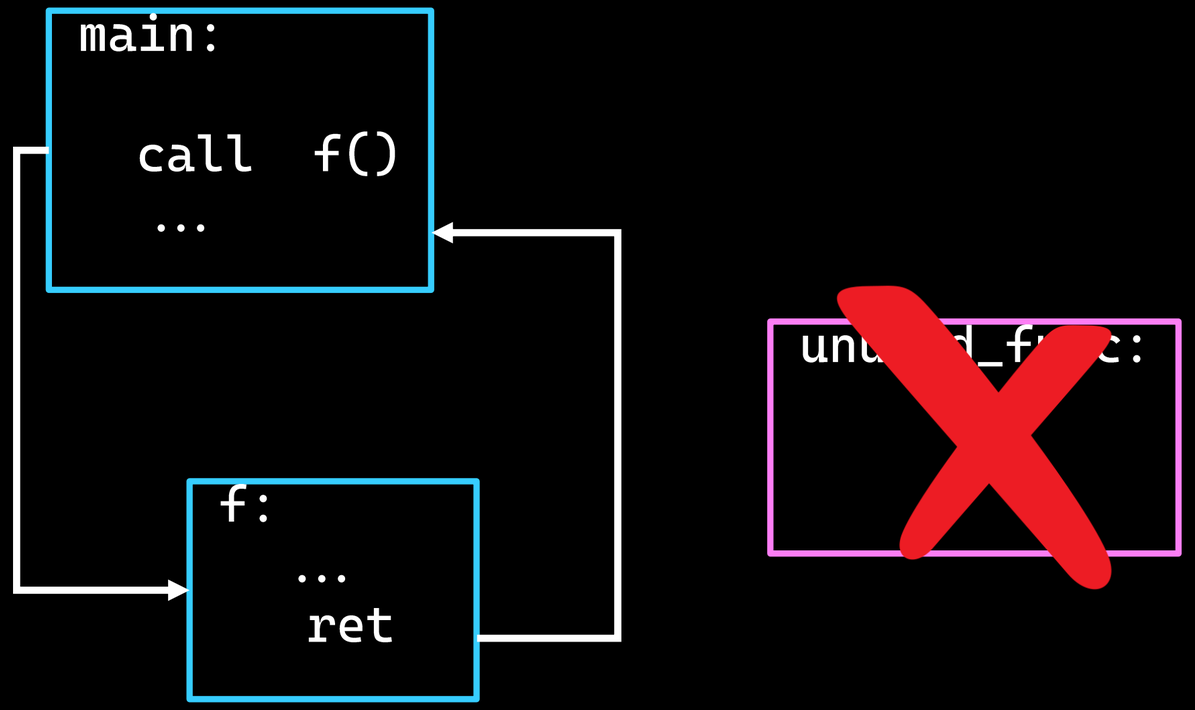
  E.g., dead code

- Available gadgets are reduced

  Attack surface shrinks

- Especially useful in libraries

  Plethora of gadgets and unused code



# Library Debloaters

- Nibbler: debloats libraries per *set of binaries*. Inserting new ones?
- Piece-Wise compilation: modifies library pages *per application* at load time
- BlankIt: copies library code at runtime to enable/disable functions

# Library Debloaters

- Nibbler: debloats libraries per *set of binaries*. Inserting new ones? → Need for recompilation
- Piece-Wise compilation: modifies library pages *per application* at load time → COW - Sharing property undermined
- BlankIt: copies library code at runtime to enable/disable functions → COW - Sharing property undermined

# Library Debloaters

- Nibbler: debloats libraries per *set of binaries*. Inserting new ones? → Need for recompilation
- Piece-Wise compilation: [illegible] → COW - Sharing property undermined
- BlankIt: copies library code at runtime to enable/disable functions → COW - Sharing property undermined

**Not practical for general-purpose systems**

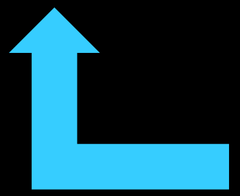
# Where we stand

- Fine(r)-grained CFI schemes not yet mature enough for widespread adoption

- Debloating not practical: unused library/module code remains exposed in applications

# Where we stand

- Fine(r)-grained CFI schemes not yet mature enough for widespread adoption
- Debloating not practical: unused library/module code remains exposed in applications
- Intel CET is the current and foreseeable main line of defense on x86



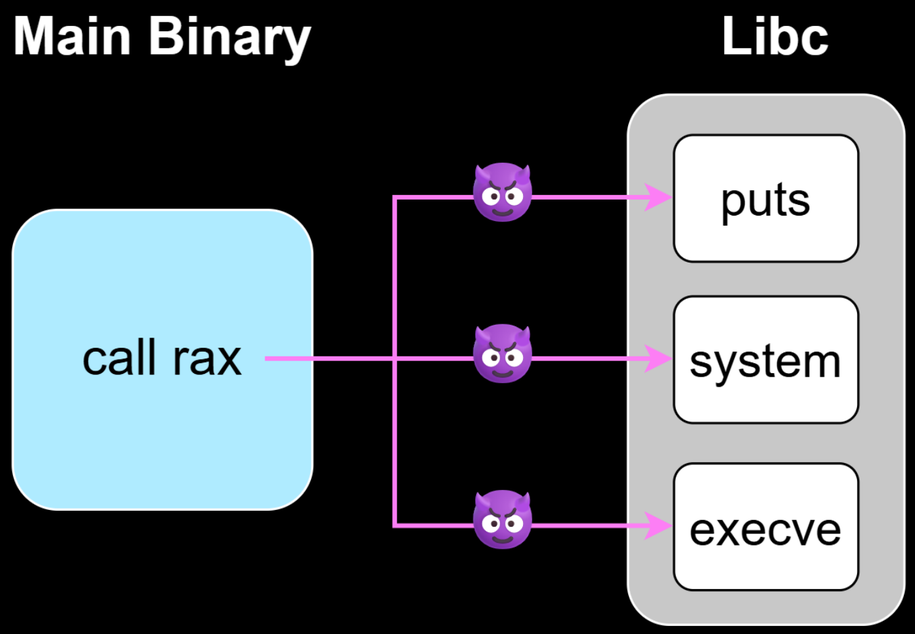
Is this that bad though?

# MOTIVATION

# CET Limitation

- Arbitrary cross-Dynamic Shared Objects (DSO) / cross-module indirect transitions are allowed
  - Leveraged by nearly every attack
  - Libraries are rich in gadgets (e.g., syscalls, sensitive functions)

Development of specialized attacks



# Function-Oriented Programming (FOP)

Class of attacks:

- Evading schemes like CET/BTI

- Entire functions as gadgets

# Function-Oriented Programming (FOP)

Class of attacks:

- Evading schemes like CET/BTI
- Entire functions as gadgets

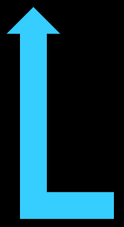
Documented attacks:

- Loop-Oriented Programming (2015)
- FOP (2018)
- Phrack’s FOP (2024)

# Function-Oriented Programming (FOP)

Class of attacks:

- Evading schemes like CET/BTI
- Entire functions as gadgets



How do they control arguments?

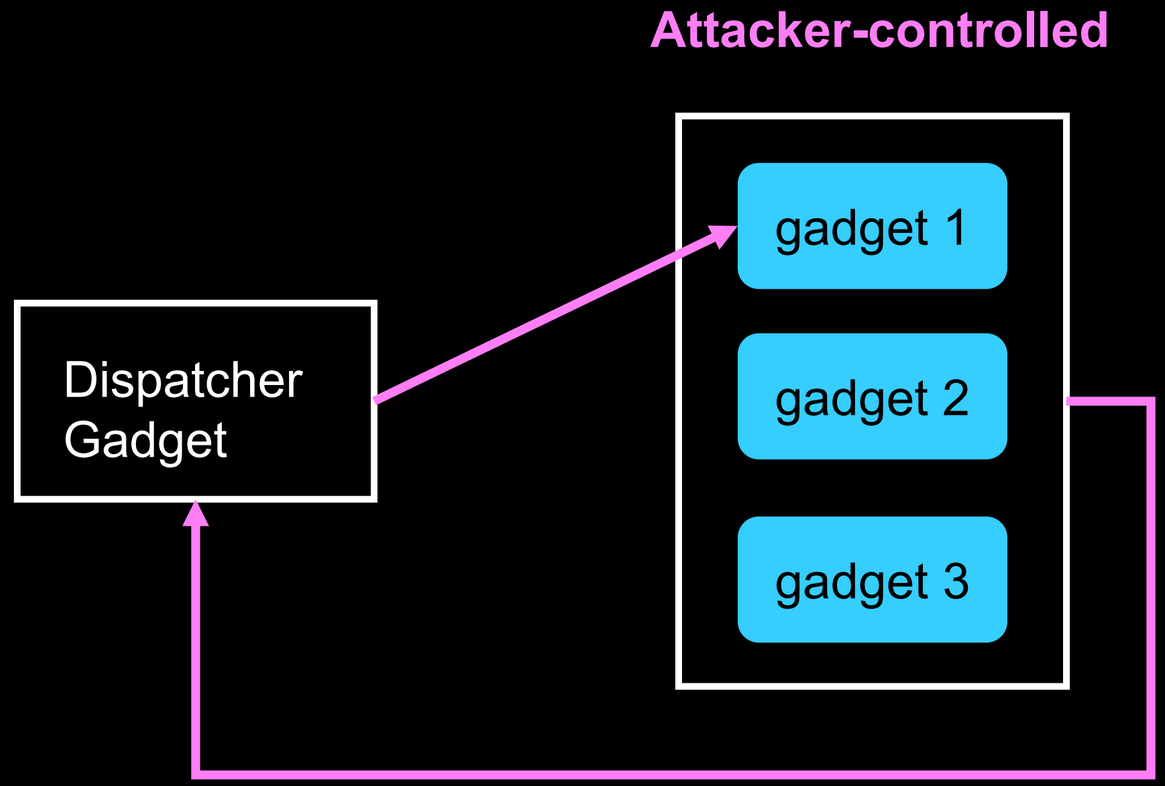
Documented attacks:

- Loop-Oriented Programming (2015)
- FOP (2018)
- Phrack’s FOP (2024)

# Dispatcher Gadget

- Orchestrator for FOP attacks
- Calls subsequent gadgets in a loop

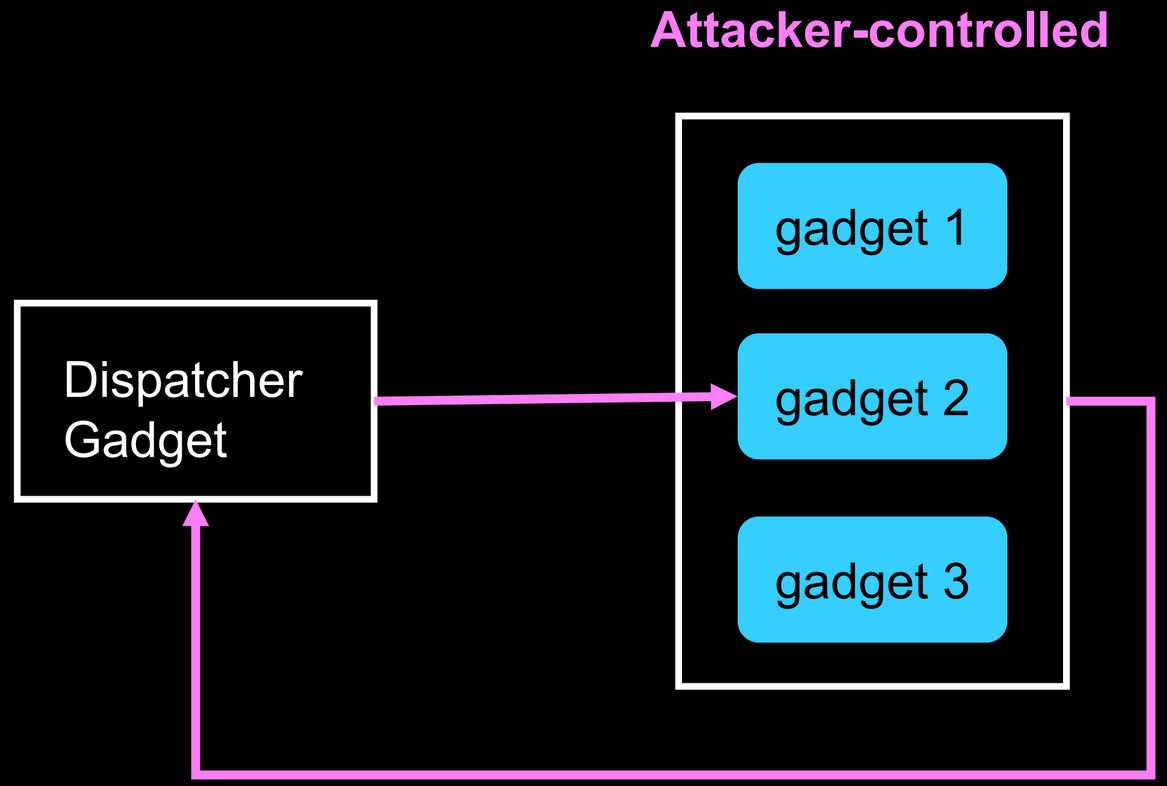
  Retrieved from attacker-controlled memory
- Conservative register usage between loops



# Dispatcher Gadget

- Orchestrator for FOP attacks
- Calls subsequent gadgets in a loop

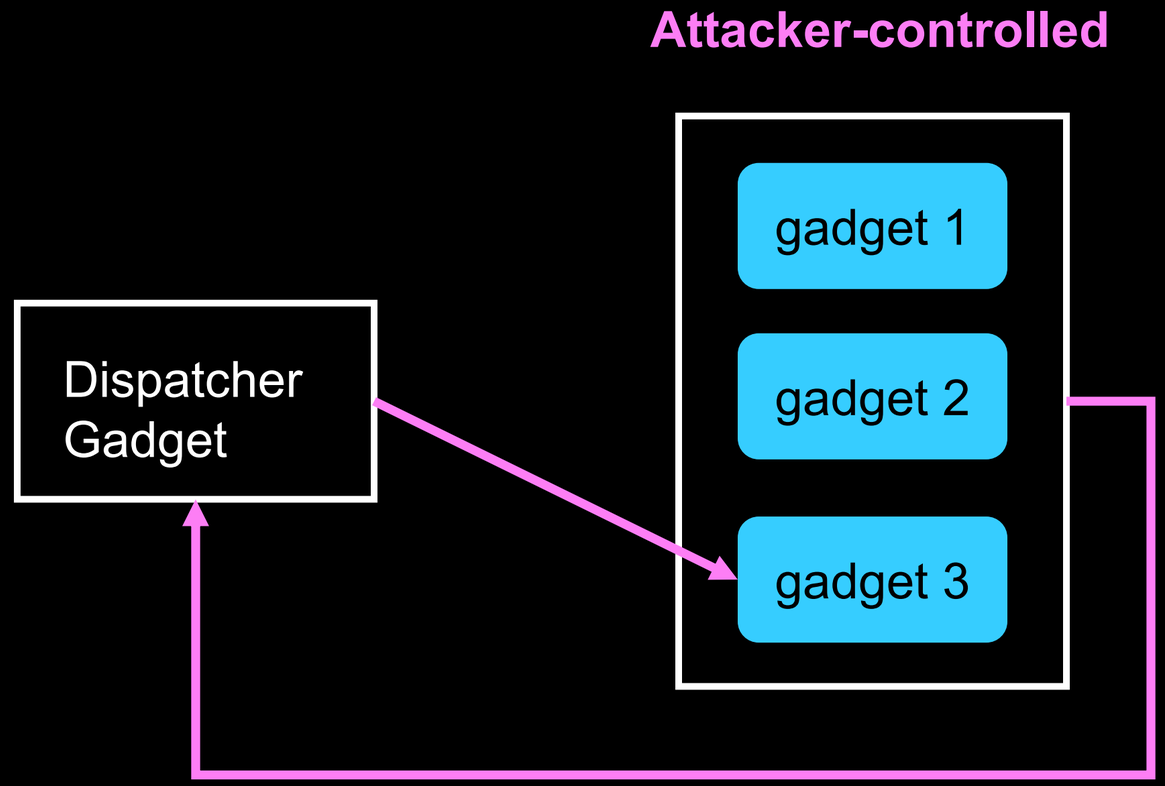
  Retrieved from attacker-controlled memory
- Conservative register usage between loops



# Dispatcher Gadget

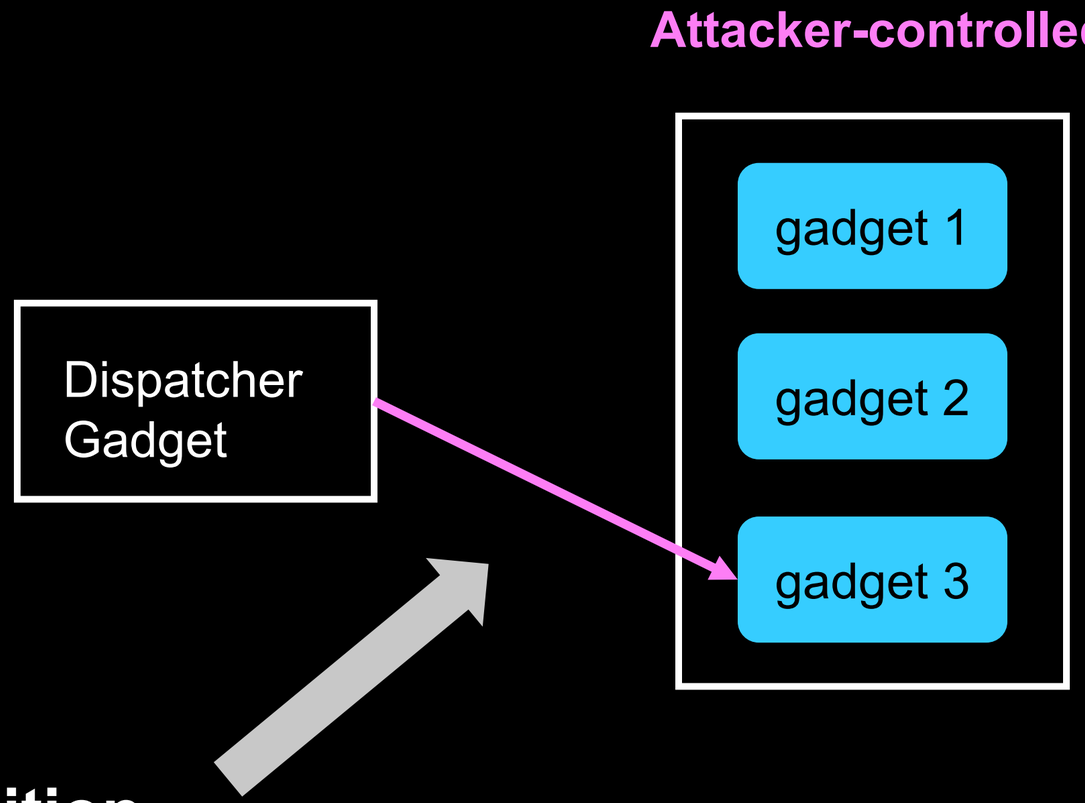
- Orchestrator for FOP attacks
- Calls subsequent gadgets in a loop

  Retrieved from attacker-controlled memory
- Conservative register usage between loops

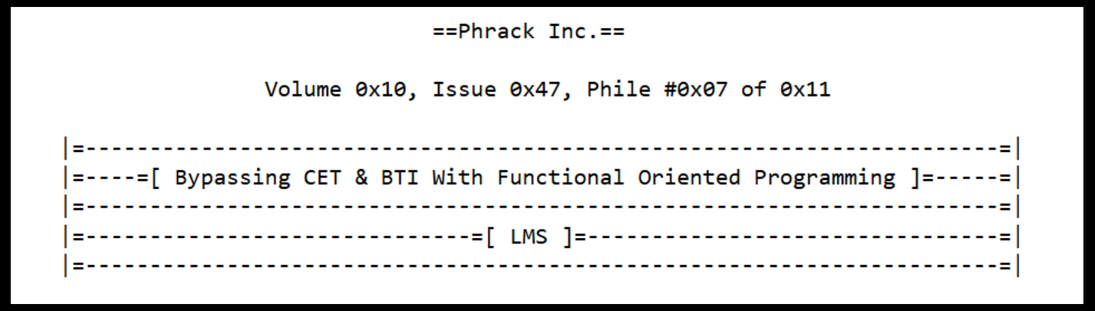


# Dispatcher Gadget

- Orchestrator for FOP attacks
- Calls subsequent gadgets in a loop
  - Retrieved from attacker-controlled memory
- Conservative register usage between loops



# Dispatcher Gadget



```
                              ==Phrack Inc.==

                  Volume 0x10, Issue 0x47, Phile #0x07 of 0x11

|=-----------------------------------------------------------------------=|
|=----=[ Bypassing CET & BTI With Functional Oriented Programming ]=-----=|
|=-----------------------------------------------------------------------=|
|=-------------------------------=[ LMS ]=--------------------------------=|
|=-----------------------------------------------------------------------=|
```

```c
void
_dl_call_fini (void *closure_map)
{
  struct link_map *map = closure_map;

  /* Make sure nothing happens if we are called twice.  */
  map->l_init_called = 0;

  ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
  if (fini_array != NULL)
    {
      ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                          + fini_array->d_un.d_ptr);
      size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                   / sizeof (ElfW(Addr)));

      while (sz-- > 0)
        ((fini_t) array[sz]) ();
    }

  /* Next try the old-style destructor.  */
  ElfW(Dyn) *fini = map->l_info[DT_FINI];
  if (fini != NULL)
    DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

dispatcher loop

# Dispatcher Gadget

- *_dl_call_fini*: part of glibc loader
- Called gadgets: part of libc

```
----|  11. Appendix B: Intel "/bin/sh" in memory chain

+--------------------------+--------------------------+
|       Function Name      |   Equivalent Operation   |
+--------------------------+--------------------------+
| _nss_files_endpwent      | MOV RDI, 0x6             |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| _dl_mcount_wrapper...    | MOV RSI, RDI             |
```

```
void
_dl_call_fini (void *closure_map)
{
  struct link_map *map = closure_map;

  /* Make sure nothing happens if we are called twice.  */
  map->l_init_called = 0;

  ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
  if (fini_array != NULL)
    {
      ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                          + fini_array->d_un.d_ptr);
      size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                   / sizeof (ElfW(Addr)));

      while (sz-- > 0)
        ((fini_t) array[sz]) ();
    }

  /* Next try the old-style destructor.  */
  ElfW(Dyn) *fini = map->l_info[DT_FINI];
  if (fini != NULL)
    DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

dispatcher loop

# Dispatcher Gadgets

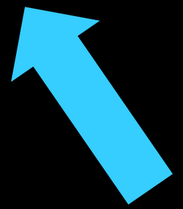
- Rare and hard to find
- Most of them not exploitable
- Exploitable ones are related to initialization and finalization routines

  LOOP attack: *_initterm()* in *msvcrt.dll*
  Phrack attack: *_dl_call_fini()* in *ld-linux*

# Dispatcher Gadgets

- Rare and hard to find
- Most of them not exploitable
- Exploitable ones are related to initialization and finalization routines

LOOP attack: *_initterm()* in *msvcrt.dll*
Phrack attack: *_dl_call_fini()* in *ld-linux*



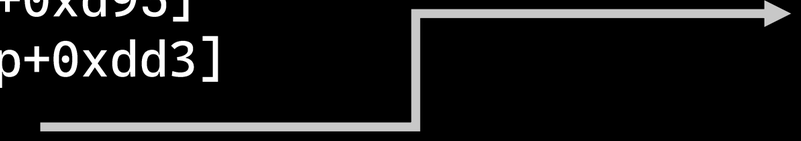
We will revisit this later

How does Linux handle cross-DSO transitions?

# Procedure Linkage Table (PLT)

- Code stubs dispatching cross-DSO calls in ELFs
- Efficient and robust
  With Full RELRO mitigation

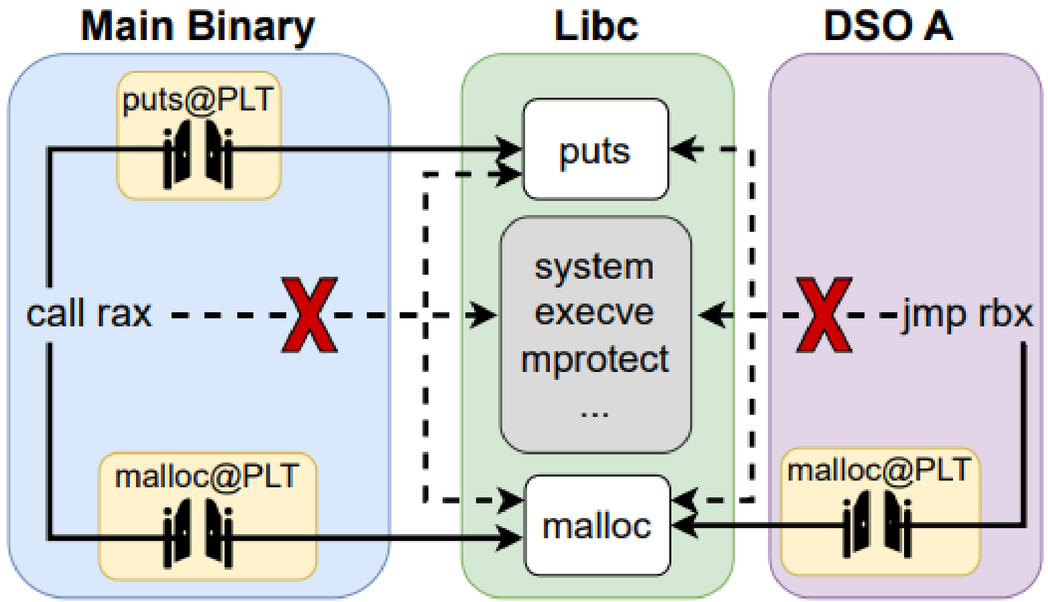
```
f():

    lea   rdi,[rip+0xd95]
    lea   rsi, [rip+0xdd3]
    call  puts@plt
    pop   rbp
    …
```



```
puts@plt:

    endbr64
    jmp   Qword Ptr[rip+0x2e6]
    nop
    …
```

# Motivation

**Why then should cross-DSO transfers be allowed outside of PLTs?**



# PLaTypus Goals

- Support both libraries and applications

  Even complex ones like *glibc* and *OpenSSL*

- Avoid library recompilation

- Secure even when some DSOs remain uninstrumented

  Third-party libraries

- Retain the sharing property of libraries

# DESIGN — METHODOLOGY - CHALLENGES

# Threat Model

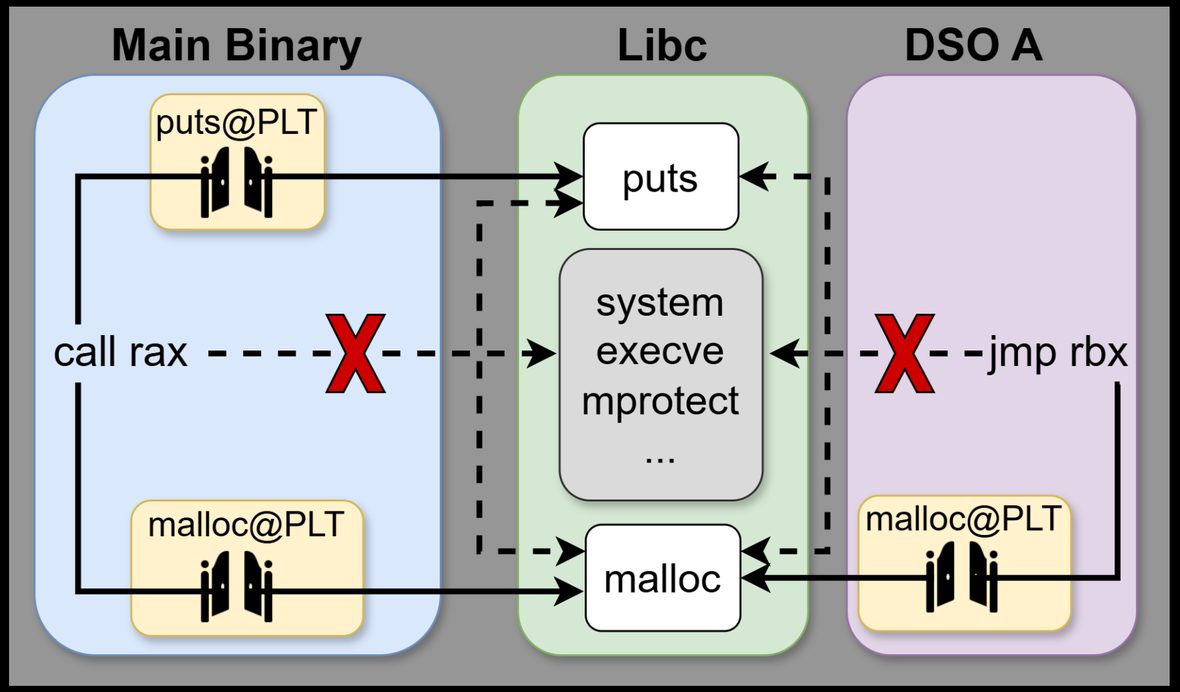
- Memory corruption vulnerabilities
  - Arbitrary reads/writes
- Function pointers can be overwritten
- Vulnerabilities can be triggered multiple times

- Intel CET in place
- Operating system enforces DEP
- Full RELRO is enabled

# PLaTypus Design

- Each DSO can only reach external functions for which it possesses PLT stubs

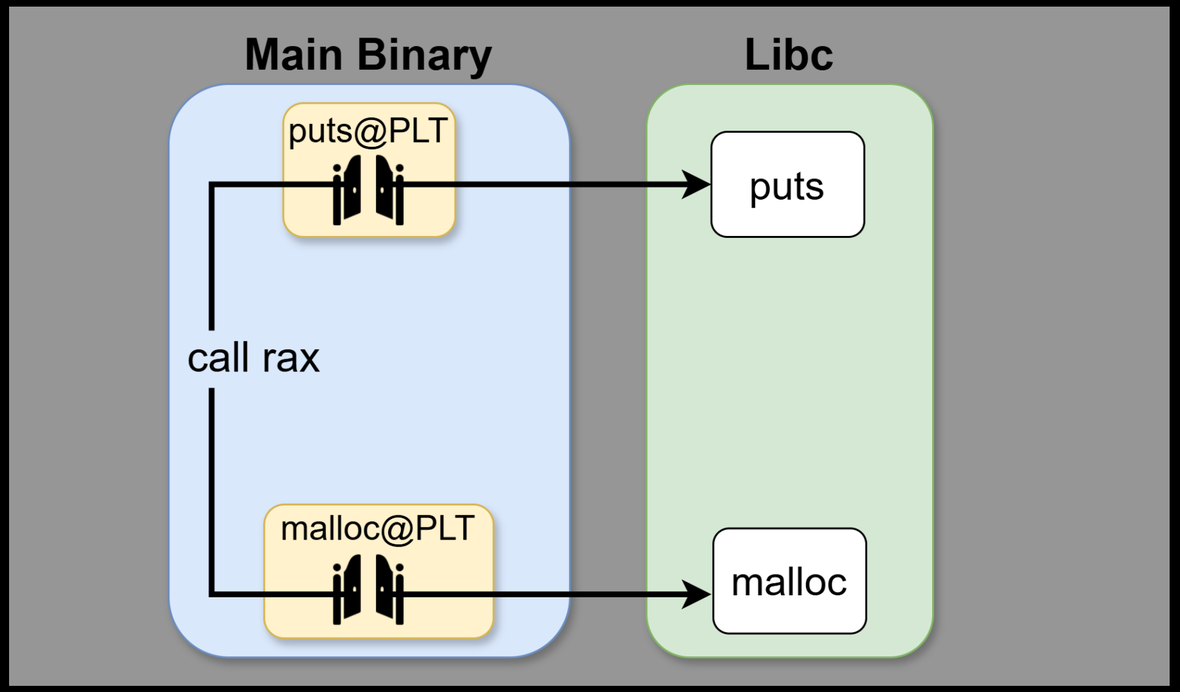
- Per-DSO granularity of enforcement



# PLaTypus Design

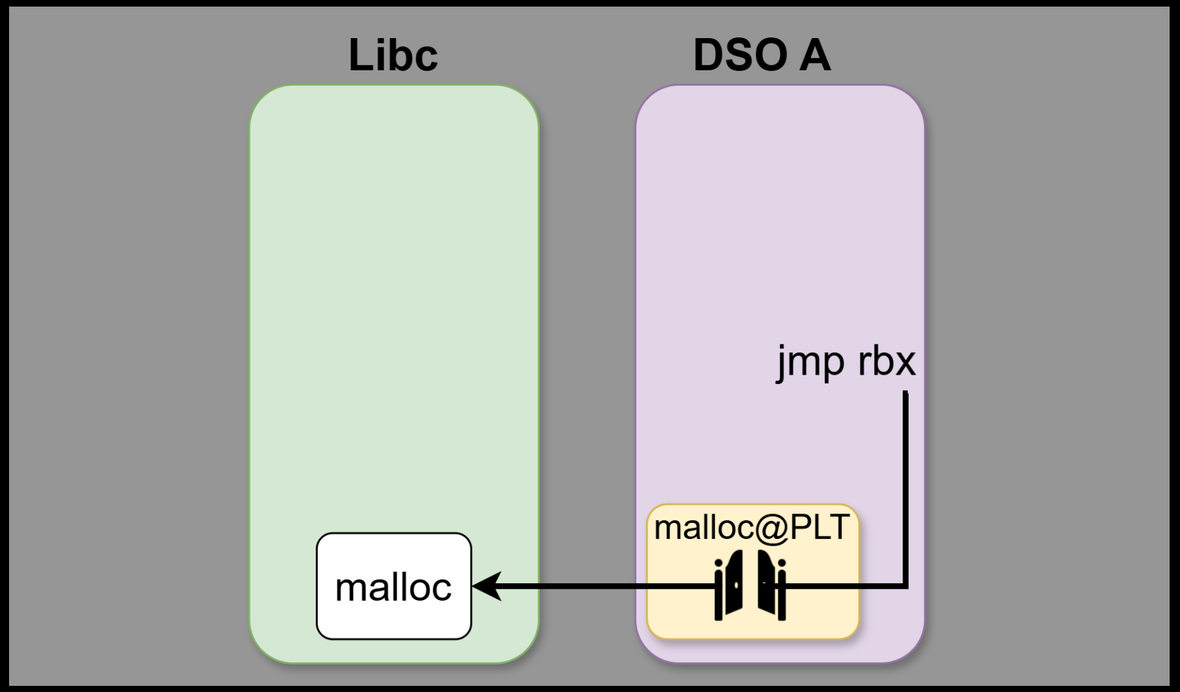
- Each DSO can only reach external functions for which it possesses PLT stubs

- Per-DSO granularity of enforcement



# PLaTypus Design

- Each DSO can only reach external functions for which it possesses PLT stubs
- Per-DSO granularity of enforcement



# Terminology

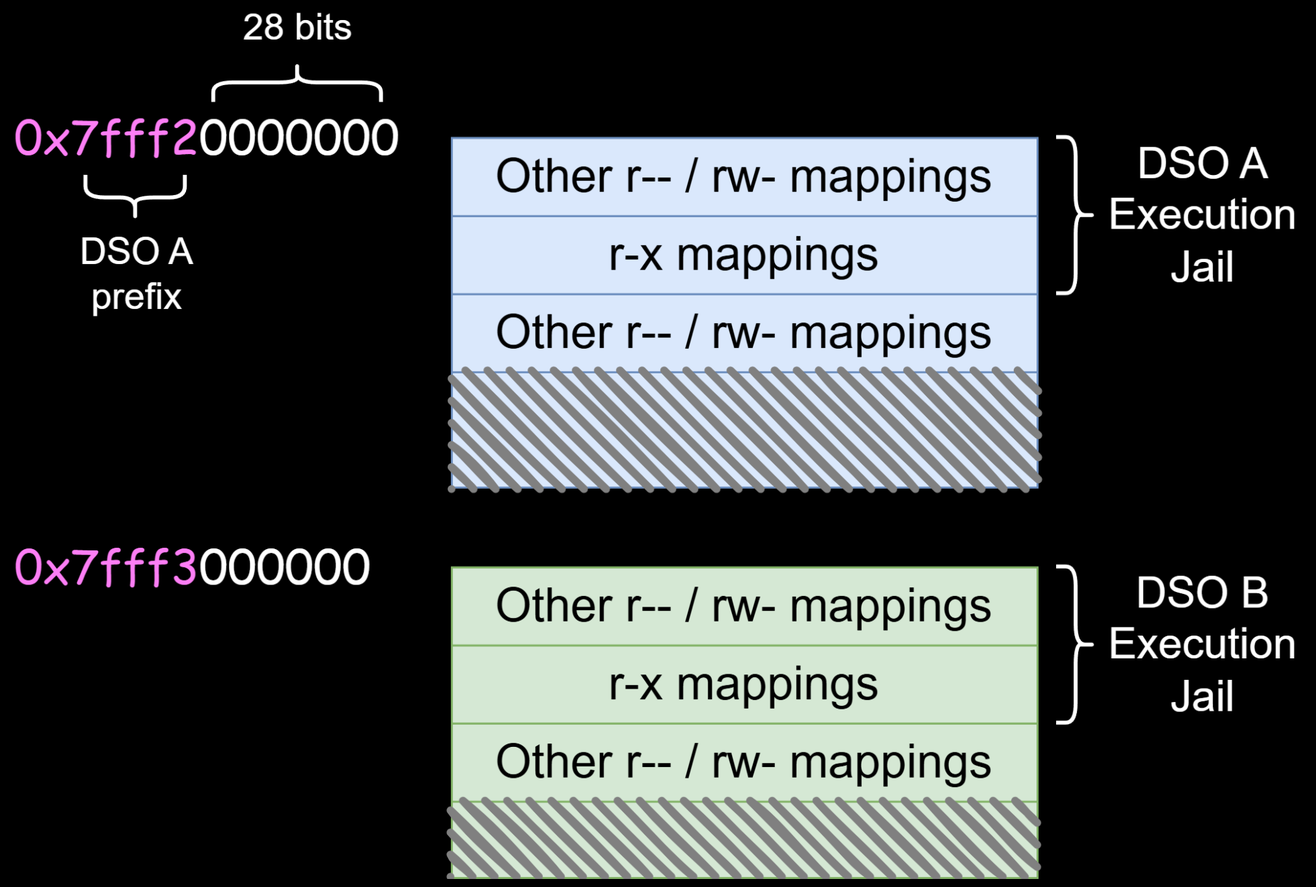
- Intra-DSO transition: *caller* and *callee* in the same module

- Inter-DSO transition: *caller* and *callee* in different modules

# Execution Jails (EJ)

- Each DSO is restricted in its EJ

  Subset of DSO's address range



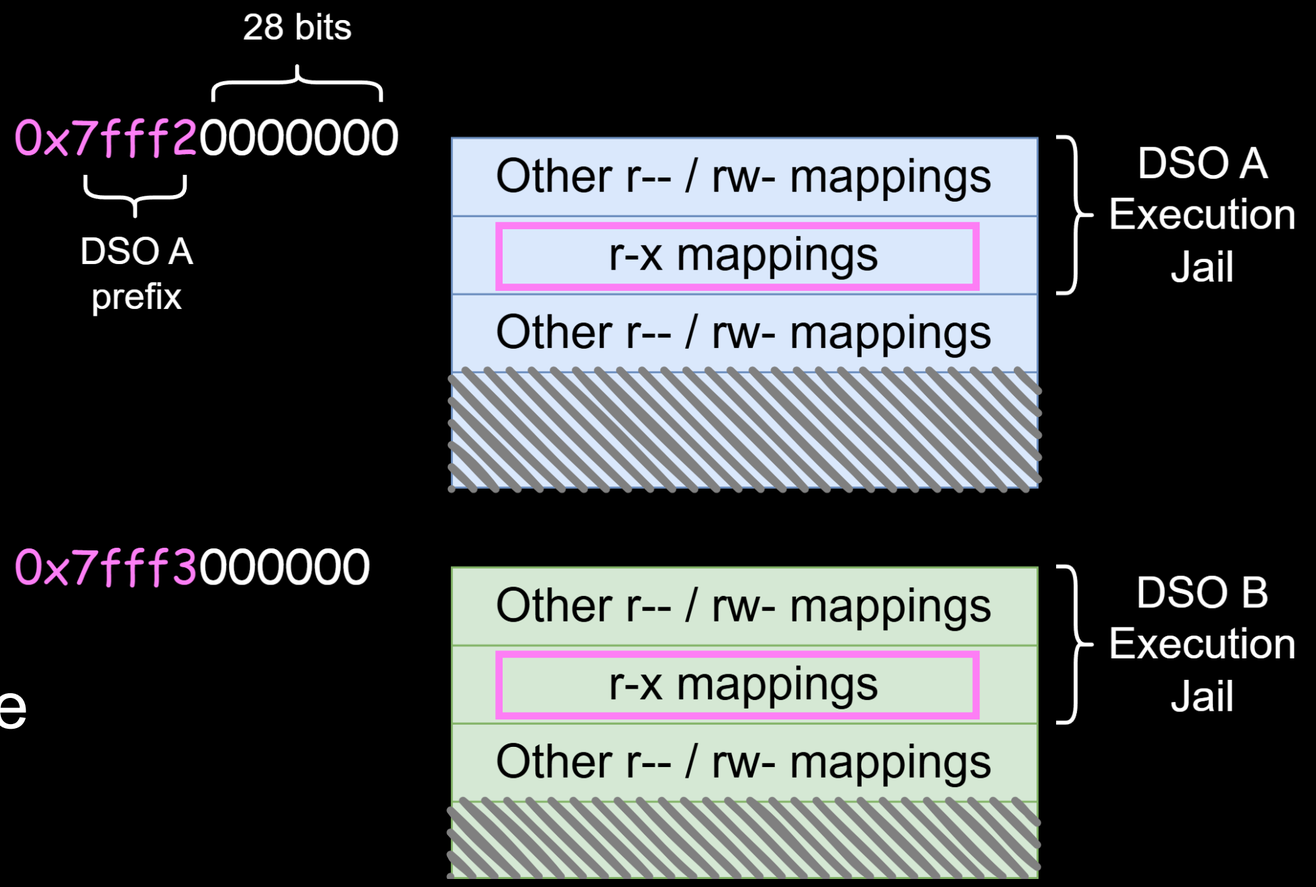
# Execution Jails (EJ)

- Each DSO is restricted in its EJ

  Subset of DSO's address range

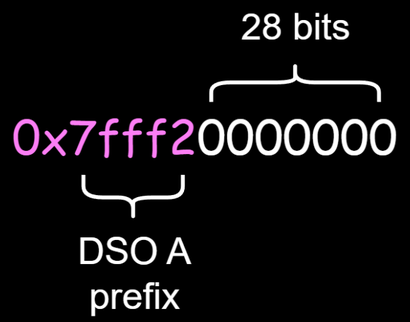
- DSO's indirect branches cannot escape the EJ

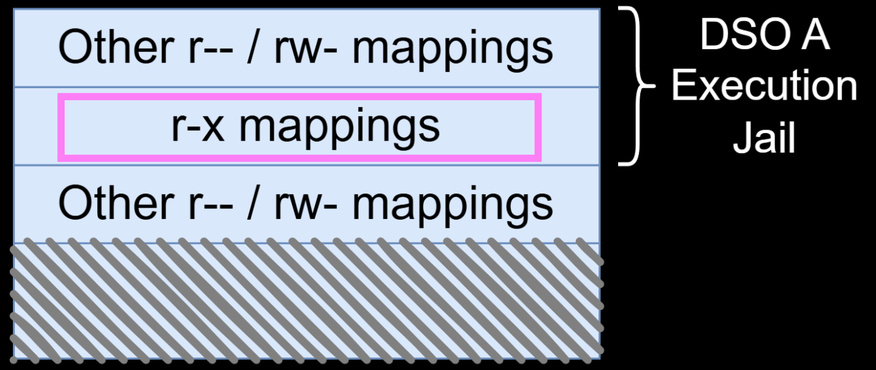
  Exception? PLT stubs



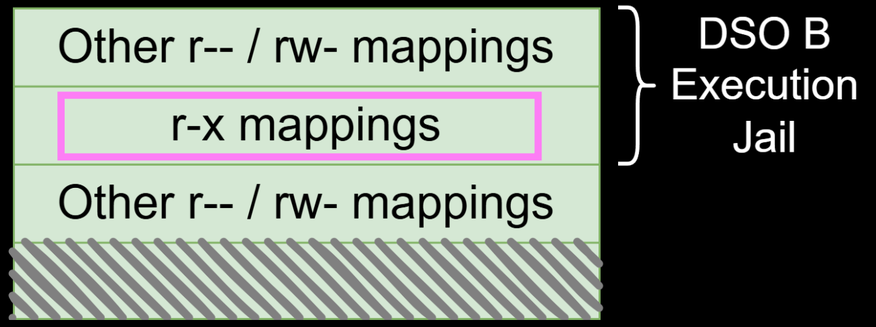
# Execution Jails (EJ) - Enforcement

- Enforced with two bitmasks



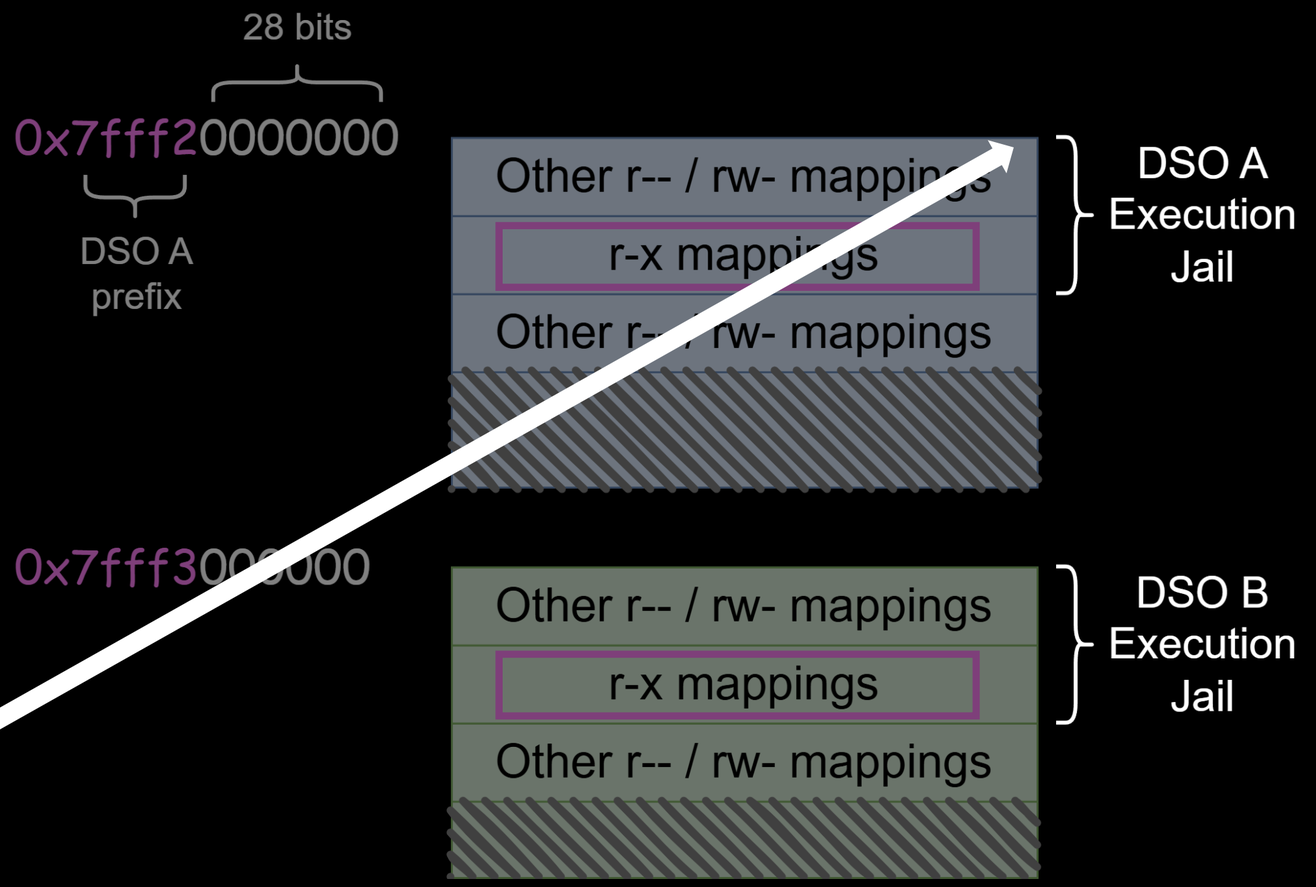

0x7fff3000000




```
; Original          ; Execution Jail

 ...                or   rax, ormask
 ...                and  rax, andmask
call rax            call rax
```

# Execution Jails (EJ) - Enforcement

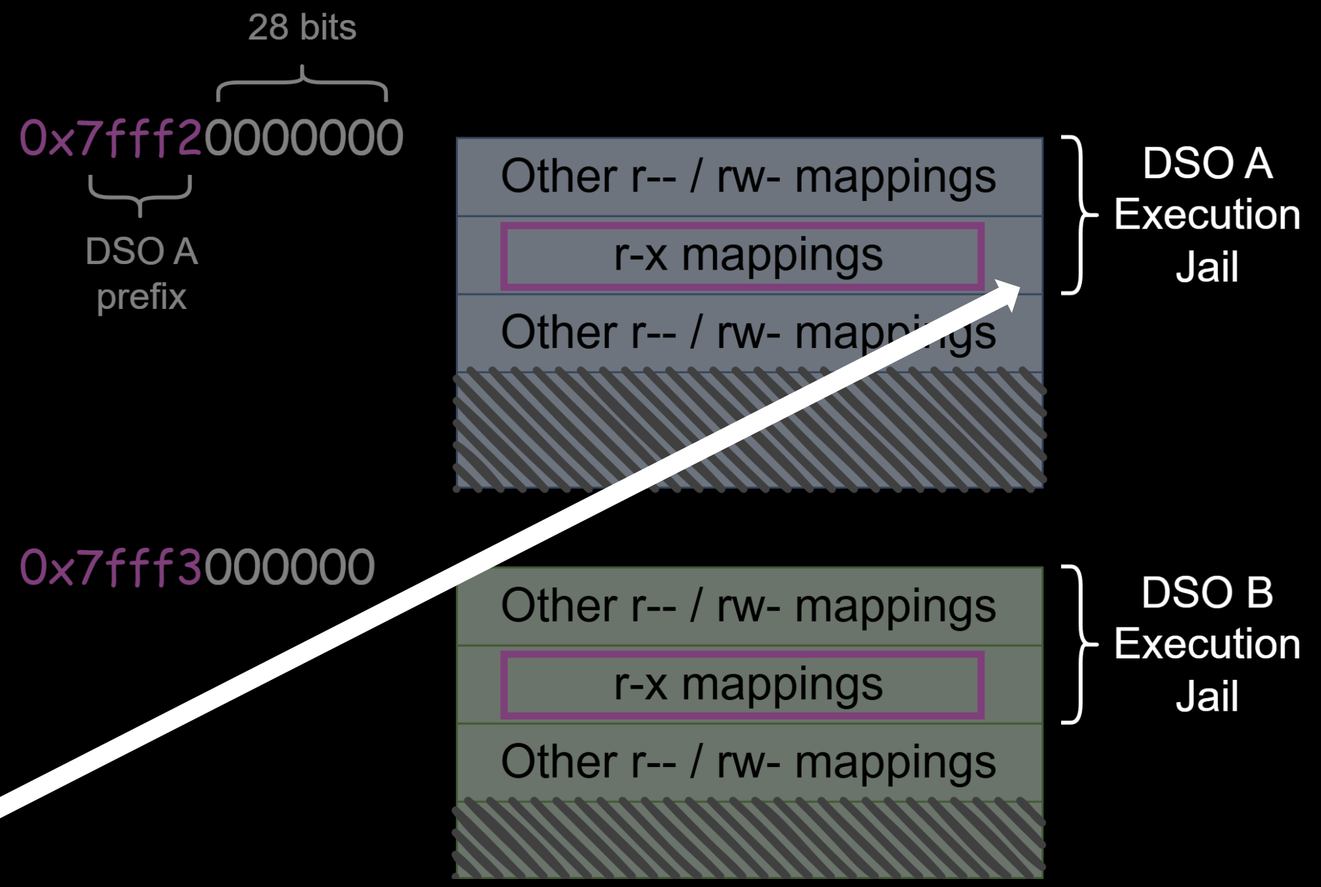
- Enforced with two bitmasks




```
; Original          ; Execution Jail

 ...                or   rax, ormask
 ...                and  rax, andmask
call rax            call rax
```

# Execution Jails (EJ) - Enforcement

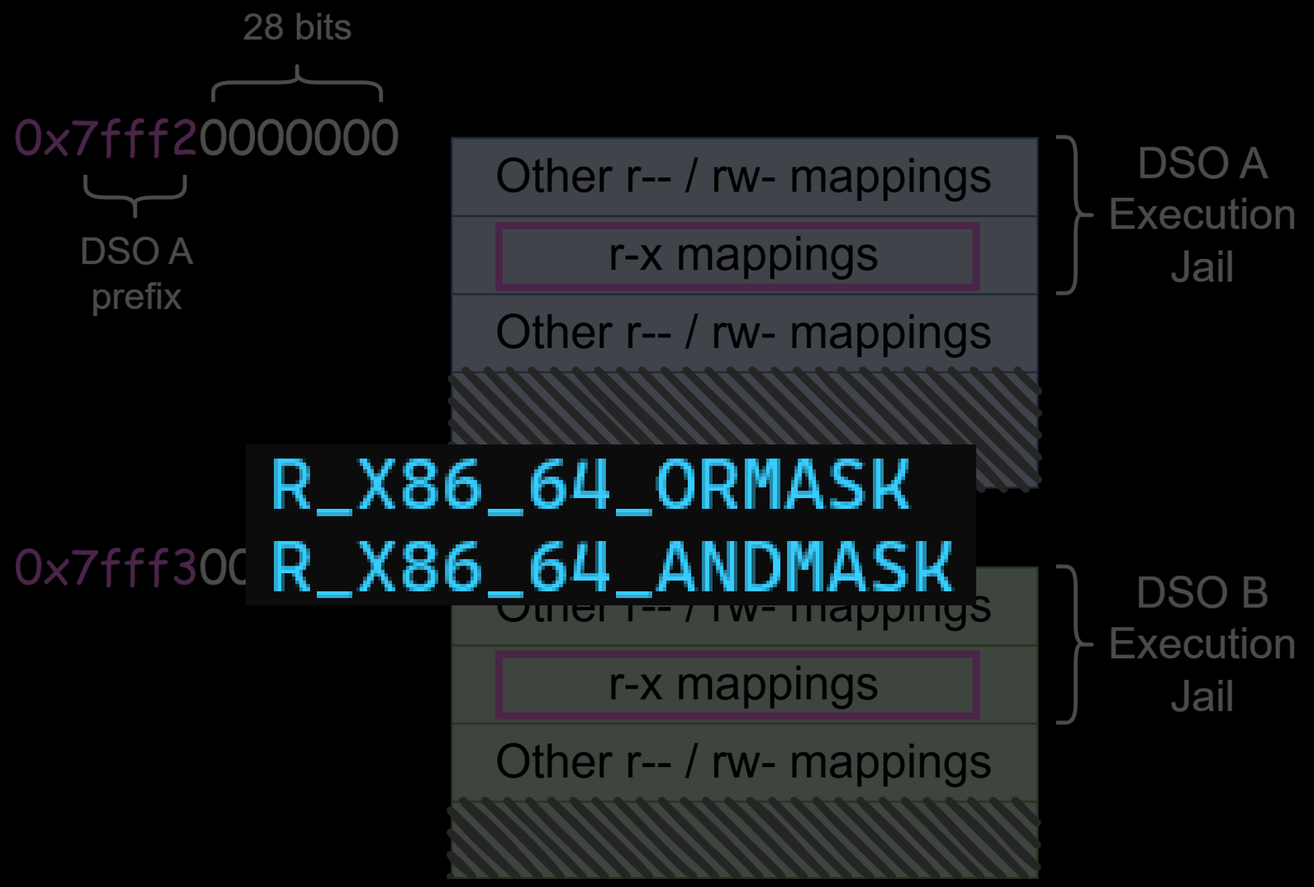
- Enforced with two bitmasks




```
; Original          ; Execution Jail

 ...                or   rax, ormask
 ...                and  rax, andmask
call rax            call rax
```

# Execution Jails (EJ)

- Enforced with two bitmasks




```
; Original          ; Execution Jail

...                 or   rax, ormask
...                 and  rax, andmask
call rax            call rax
```

# Execution Jails (EJ)

```
0x0000555555555dd1 <+81>:    mov    rcx,QWORD PTR [rip+0x12a0]        # 0x555555557078
0x0000555555555dd8 <+88>:    or     rax,rcx
0x0000555555555ddb <+91>:    mov    rcx,QWORD PTR [rip+0x129e]        # 0x555555557080
0x0000555555555de2 <+98>:    and    rax,rcx
0x0000555555555de5 <+101>:   call   rax
0x0000555555555de7 <+103>:   xor    eax,eax
```

# Execution Jails (EJ)

Inter-DSO transitions are transformed to intra-DSO

# Non-PLT Relocations

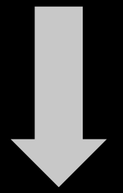
- Not all imports go through the PLT…

# Non-PLT Relocations

- Not all imports go through the PLT…

  *Address-taken* symbols

```c
int main(void) {
        int (*fp)(const char *) = &puts;
        fp("hello");
}
```



-O0

```
R_X86_64_GLOB_DAT  puts@GLIBC_2.2.5 + 0
```

# Non-PLT Relocations

```
Dump of assembler code for function main:
   0x0000555555555130 <+0>:     push   rbp
   0x0000555555555131 <+1>:     mov    rbp,rsp
   0x0000555555555134 <+4>:     sub    rsp,0x10
   0x0000555555555138 <+8>:     mov    DWORD PTR [rbp-0x4],0x0
   0x000055555555513f <+15>:    mov    rax,QWORD PTR [rip+0x2e82]        # 0x555555557fc8
   0x0000555555555146 <+22>:    mov    QWORD PTR [rbp-0x10],rax
   0x000055555555514a <+26>:    lea    rdi,[rip+0xeb3]        # 0x555555556004
   0x0000555555555151 <+33>:    call   QWORD PTR [rbp-0x10]
   0x0000555555555154 <+36>:    xor    eax,eax
   0x0000555555555156 <+38>:    add    rsp,0x10
   0x000055555555515a <+42>:    pop    rbp
   0x000055555555515b <+43>:    ret
```

# Non-PLT Relocations

```
Dump of assembler code for function main:
   0x0000555555555130 <+0>:     push   rbp
   0x0000555555555131 <+1>:     mov    rbp,rsp
   0x0000555555555134 <+4>:     sub    rsp,0x10
   0x0000555555555138 <+8>:     mov    DWORD PTR [rbp-0x4],0x0
   0x000055555555513f <+15>:    mov    rax,QWORD PTR [rip+0x2e82]        # 0x555555557fc8
   0x0000555555555146 <+22>:    mov    QWORD PTR [rbp-0x10],rax
   0x000055555555514a <+26>:    lea    rdi,[rip+0xeb3]        # 0x555555556004
   0x0000555555555151 <+33>:    call   QWORD PTR [rbp-0x10]
   0x0000555555555154 <+36>:    xor    eax,eax
   0x0000555555555156 <+38>:    add    rsp,0x10
   0x000055555555515a <+42>:    pop    rbp
   0x000055555555515b <+43>:    ret
```

Instrumentation here would corrupt the cross-DSO pointer…

# Fake PLTs

For such symbols:

1. Emit PLT stubs

   New section: *.fakeplt.sec*

2. Redirect associated relocations to point to these stubs

# Fake PLTs

For such symbols:

1. Emit PLT stubs

   New section: *.fakeplt.sec*

2. Redirect associated relocations to point to these stubs

```
puts@plt:

0x1860:    endbr64
           jmp   Qword Ptr[rip+0x2e6]
           nop
```

```
int main(void) {

        int (*fp)(const char *) = &puts;

        fp("hello");
}
```

# Fake PLTs

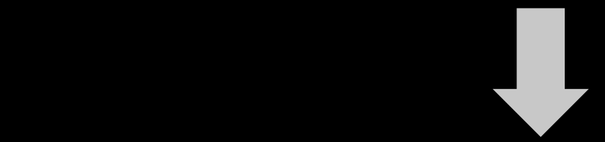
For such symbols:

1. Emit PLT stubs

   New section: *.fakeplt.sec*

2. Redirect associated relocations to point to these stubs

```
int main(void) {

        int (*fp)(const char *) = &puts;

        fp("hello");
}
```



```
     Type                Symbol's Value
R_X86_64_RELATIVE        1860
```

```
puts@plt:

0x1860:    endbr64
           jmp    Qword Ptr[rip+0x2e6]
           nop
```

# Fake PLTs

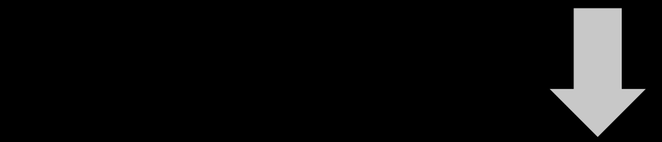
For such symbols:

1. Emit PLT stubs

   New section: *.fakeplt.sec*

2. Redirect associated relocations to point to these stubs

```
int main(void) {

        int (*fp)(const char *) = &puts;

        fp("hello");
}
```



```
     Type                 Symbol's Value
R_X86_64_RELATIVE         1860
```

```
R_X86_64_JUMP_SLOT   puts@GLIBC_2.2.5 + 0
```

```
puts@plt:

0x1860:    endbr64
           jmp    Qword Ptr[rip+0x2e6]
           nop
```

# Fake PLTs

```
0x55554554fbf0:      endbr64
0x55554554fbf4:      push   rbp
0x55554554fbf5:      mov    rbp,rsp
0x55554554fbf8:      sub    rsp,0x10
0x55554554fbfc:      mov    DWORD PTR [rbp-0x4],0x0
0x55554554fc03:      mov    rax,QWORD PTR [rip+0x12de]        # 0x555545550ee8
0x55554554fc0a:      mov    QWORD PTR [rbp-0x10],rax
0x55554554fc0e:      mov    rax,QWORD PTR [rbp-0x10]
0x55554554fc12:      mov    rcx,QWORD PTR [rip+0x12d7]        # 0x555545550ef0
0x55554554fc19:      or     rax,rcx
0x55554554fc1c:      mov    rcx,QWORD PTR [rip+0x12d5]        # 0x555545550ef8
0x55554554fc23:      and    rax,rcx
0x55554554fc26:      lea    rdi,[rip+0xffffffffffffedff]        # 0x55554554ea2c
0x55554554fc2d:      call   rax
0x55554554fc2f:      xor    eax,eax
0x55554554fc31:      add    rsp,0x10
0x55554554fc35:      pop    rbp
0x55554554fc36:      ret
```

# Fake PLTs

```
0x55554554fbf0:      endbr64
0x55554554fbf4:      push   rbp
0x55554554fbf5:      mov    rbp,rsp
0x55554554fbf8:      sub    rsp,0x10
0x55554554fbfc:      mov    DWORD PTR [rbp-0x4],0x0
0x55554554fc03:      mov    rax,QWORD PTR [rip+0x12de]        # 0x555545550ee8
0x55554554fc0a:      mov    QWORD PTR [rbp-0x10],rax
0x55554554fc0e:      mov    rax,QWORD PTR [rbp-0x10]
0x55554554fc12:      mov    rcx,QWORD PTR [rip+0x12d7]        # 0x555545550ef0
0x55554554fc19:      or     rax,rcx
0x55554554fc1c:      mov    rcx,QWORD PTR [rip+0x12d5]        # 0x555545550ef8
0x55554554fc23:      and    rax,rcx
0x55554554fc26:      lea    rdi,[rip+0xffffffffffffedff]        # 0x55554554ea2c
0x55554554fc2d:      call   rax
0x55554554fc2f:      xor    eax,eax
0x55554554fc31:      add    rsp,0x10
0x55554554fc35:      pop    rbp
0x55554554fc36:      ret
```

```
gef➤  x/gx 0x555545550ee8
0x555545550ee8: 0x000055554554fcd0
```

# Fake PLTs

```
0x55554554fbf0:      endbr64
0x55554554fbf4:      push   rbp
0x55554554fbf5:      mov    rbp,rsp
0x55554554fbf8:      sub    rsp,0x10
0x55554554fbfc:      mov    DWORD PTR [rbp-0x4],0x0
0x55554554fc03:      mov    rax,QWORD PTR [rip+0x12de]        # 0x555545550ee8
0x55554554fc0a:      mov    QWORD PTR [rbp-0x10],rax
0x55554554fc0e:      mov    rax,QWORD PTR [rbp-0x10]
0x55554554fc12:      mov    rcx,QWORD PTR [rip+0x12d7]        # 0x555545550ef0
0x55554554fc19:      or     rax,rcx
0x55554554fc1c:      mov    rcx,QWORD PTR [rip+0x12d5]        # 0x555545550ef8
0x55554554fc23:      and    rax,rcx
0x55554554fc26:      lea    rdi,[rip+0xffffffffffffedff]        # 0x55554554ea2c
0x55554554fc2d:      call   rax
0x55554554fc2f:      xor    eax,eax
0x55554554fc31:      add    rsp,0x10
0x55554554fc35:      pop    rbp
0x55554554fc36:      ret
```

```
gef➤  x/gx 0x555545550ee8
0x555545550ee8: 0x000055554554fcd0
```

```
gef➤  x/3i 0x000055554554fcd0
   0x55554554fcd0:      endbr64
   0x55554554fcd4:      jmp    QWORD PTR [rip+0x124e]        # 0x555545550f28
   0x55554554fcda:      nop    WORD PTR [rax+rax*1+0x0]
```

# Fake vs Normal PLTs

- Normal PLTs are reached through **direct** calls

- Fake PLTs are reached through **indirect** calls

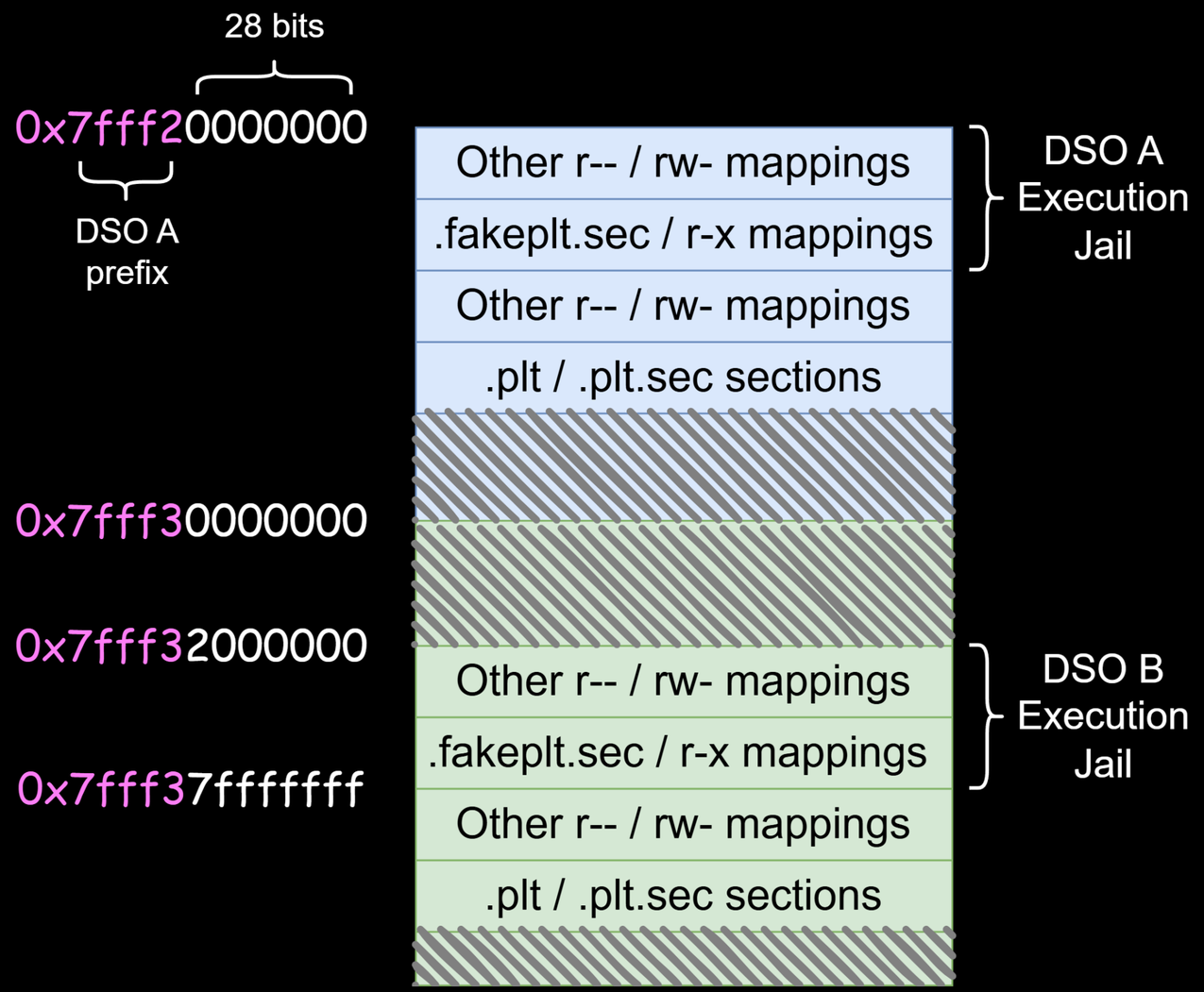
# Fake vs Normal PLTs

- Normal PLTs are reached through **direct** calls

- Fake PLTs are reached through **indirect** calls

We can place Normal PLTs **outside** each DSO’s Execution Jail

# Final Design



# Quick Summary

Hijacked pointers can reach:

1. Intra-module functions

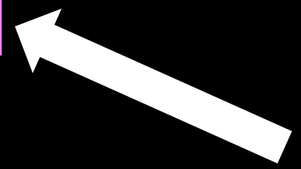
2. Only the Fake PLT stubs of the module

# Quick Summary

Hijacked pointers can reach:

1. Intra-module functions

2. Only the Fake PLT stubs of the module



Possible cross-DSO targets

# Tooling

## LLVM toolchain (20.1.0)

- Modifications to the compiler and the linker
- Instrumentation through compiler passes

## Glibc (2.41)

- Modifications to the loader

# Challenges

- Callbacks

# Challenges

- Callbacks

- Dynamically loaded modules (via *dlopen*)

# Callbacks

**Module A**

```
void f1() {
    ...
    qsort(arr,n,sizeof(int),compare)
    ...
}
```

```
void compare( ... ) {
    ...
}
```

# Callbacks

**Module A**

```
void f1() {
    ...
    qsort(arr,n,sizeof(int),compare)
    ...
}
```

```
void compare( ... ) {
    ...
}
```

**Libc**

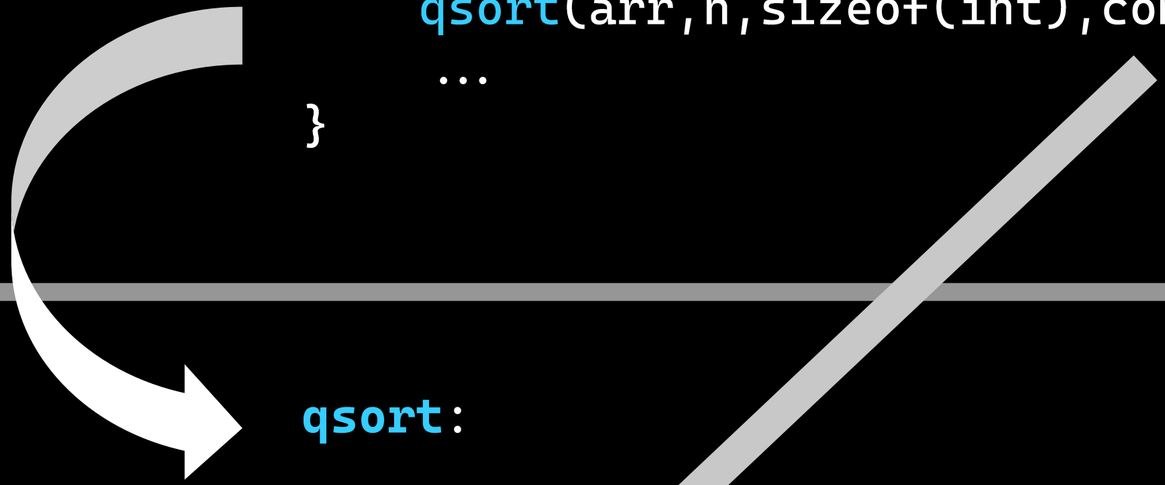
```
qsort:
    ...
    or   rcx,ormask
    and  rcx,andmask
    call rcx
    ...
```

# Callbacks

**Module A**

```
void f1() {
    ...
    qsort(arr,n,sizeof(int),compare)
    ...
}
```

```
void compare( ... ) {
    ...
}
```



**Libc**

```
qsort:
    ...
    or   rcx,ormask
    and  rcx,andmask
    call rcx
    ...
```

# Callbacks

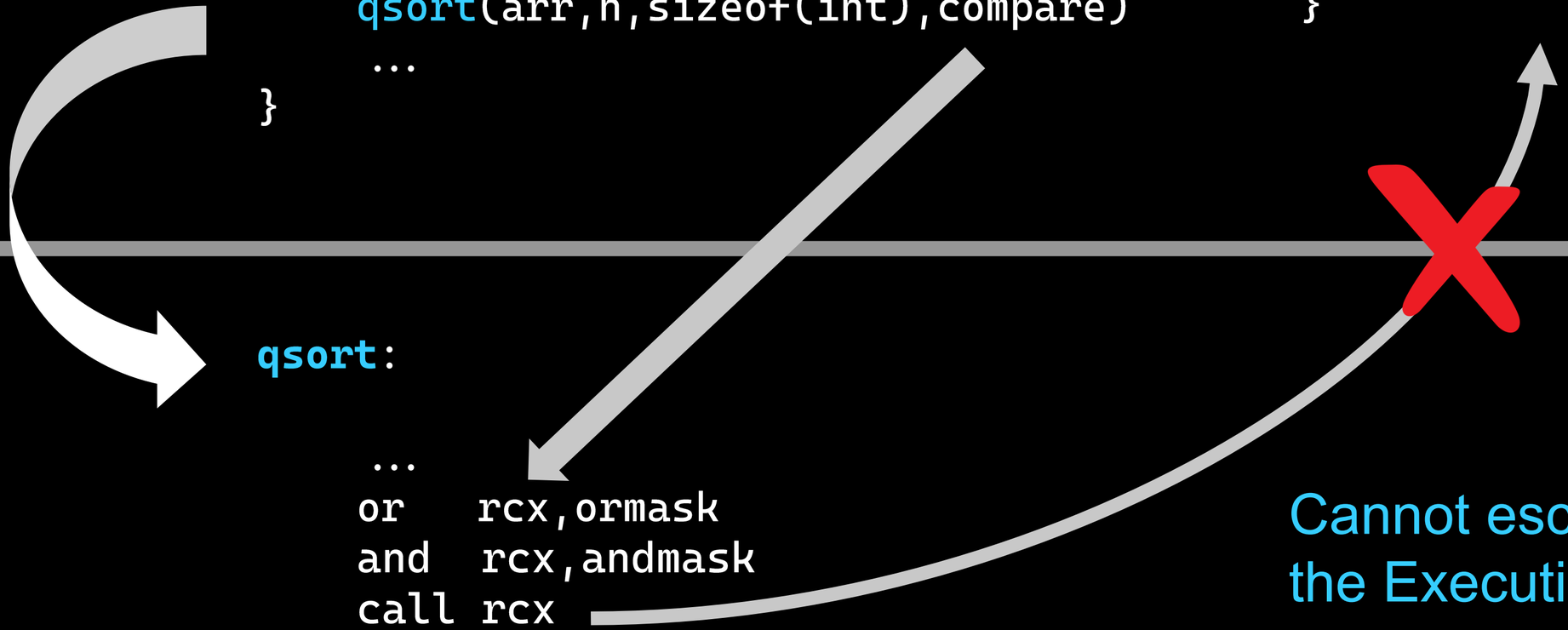
**Module A**

```
void f1() {
    ...
    qsort(arr,n,sizeof(int),compare)
    ...
}
```

```
void compare( ... ) {
    ...
}
```

**Libc**

```
qsort:
    ...
    or   rcx,ormask
    and  rcx,andmask
    call rcx
    ...
```



Cannot escape
the Execution Jail

# Handling Callbacks

1. Collect callback symbols (per module)

2. Instrument callback sites with extended masking

   Places where callbacks are invoked

# Symbol Gathering

- Leverage 2 compiler passes

  Examining Intermediate Representation

```
void f1() {
     ...
    qsort(arr,n,sizeof(int),compare1)
    qsort(arr,n,sizeof(int),compare2)
     ...
}
```

# Symbol Gathering

- Leverage 2 compiler passes

  Examining Intermediate Representation

- Create callback tables

  One for each DSO dependency

```
void f1() {
     ...
    qsort(arr,n,sizeof(int),compare1)
    qsort(arr,n,sizeof(int),compare2)
     ...
}
```

```
{
  “libc”: [compare1, compare2],
  “libcrypto”: [ ... ],
   ...
}
```

# Symbol Gathering

- Leverage 2 compiler passes

  Examining Intermediate Representation

```
void f1() {
     ...
    qsort(arr,n,sizeof(int),compare1)
    qsort(arr,n,sizeof(int),compare2)
     ...
}
```

- Create callback tables

  One for each DSO dependency

```
{
  “libc”: [compare1, compare2],
  “libcrypto”: [ ... ],
   ...
}
```

If the current module is used, then *libc* may need to call *compare1* and *compare2*

# Extended Masking

- Instrument callback sites

```
; Original:
   call  rax

; Extended Masking:
   mov   r11,rax
   or    rax,ormask
   and   rax,andmask
   cmp   r11,rax
   jne   call_target
   call  rax
   jmp   after_call

call_target:
   call  DSO_callback_table

after_call:
 ...
```

# Extended Masking

- Instrument callback sites

If not equal, then target outside the Execution Jail

```
; Original:
   call  rax

; Extended Masking:
   mov   r11,rax
   or    rax,ormask
   and   rax,andmask
   cmp   r11,rax
   jne   call_target
   call  rax
   jmp   after_call

call_target:
   call  DSO_callback_table

after_call:
 ...
```

# Extended Masking

- Instrument callback sites

If not equal, then target outside the Execution Jail

Search target inside the callback table of the DSO

```
; Original:
   call  rax

; Extended Masking:
   mov   r11,rax
   or    rax,ormask
   and   rax,andmask
   cmp   r11,rax
   jne   call_target
   call  rax
   jmp   after_call

call_target:
   call  DSO_callback_table

after_call:
 ...
```

# Extended Masking

- Instrument callback sites

If not equal, then target outside the Execution Jail

Search target inside the callback table of the DSO

Not found? Halt execution 😈 😈

```
; Original:
   call  rax

; Extended Masking:
   mov   r11,rax
   or    rax,ormask
   and   rax,andmask
   cmp   r11,rax
   jne   call_target
   call  rax
   jmp   after_call

call_target:
   call  DSO_callback_table

after_call:
 ...
```

# Special Callbacks

1. main()

   Called by libc

2. Initialization and finalization routines

   *.init, .fini, .init_array, .fini_array* sections

3. Exit handlers

   Registered via *atexit()*

# Special Callbacks

1. main()

   Called by libc

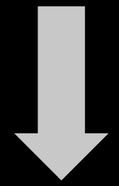
2. Initialization and finalization routines

   *.init, .fini, .init_array, .fini_array* sections

3. Exit handlers

   Registered via *atexit()*

Complete set of these callback routines known after compilation and linking



Create callback tables just for them

# Dynamically Loaded Modules

Typical approaches:

- On-the-fly call target recalculation
- Rewriting of code pages
- Profiling applications with benchmarks

# Dynamically Loaded Modules

Typical approaches:

- On-the-fly call target recalculation
- Rewriting of code pages
- Profiling applications with benchmarks

# Dynamically Loaded Modules

- Preload necessary modules at startup
- Generate appropriate PLT stubs for *dlsym-ed* symbols
- Most libraries do not use *dlopen()*

  Libc frequently loads NSS libraries

# EVALUATION

# Evaluation Metrics

1. Correctness

   Does PLaTypus instrumentation preserve program behavior?

2. Security guarantees

   What is the reduction in cross-DSO gadgets?
   Are state-of-the-art CET bypass attacks mitigated?

3. Performance

   What is the runtime overhead introduced by PLaTypus?

# Correctness

- No failures observed in test suites of 19 real-world applications

Table 1. BENCHMARK SUITE. EVERY BENCHMARK IS FOLLOWED BY ITS VERSION NUMBER.

| | | | | |
|---|---|---|---|---|
| rm v9.7 | mkdir v9.7 | gzip v1.14 | make v4.4.1 | Nginx v1.28.0 |
| date v9.7 | sort v9.7 | grep v3.12 | lighttpd v1.4.79 | Redis v8.2.0 |
| uniq v9.7 | tar v1.35 | objdump v2.44 | wget v1.25.0 | SQLite v3.50.4 |
| chown v9.7 | bzip2 v1.0.8 | bftpd v6.3 | memcached v1.6.38 | |

# Correctness

- No failures observed in test suites of 19 real-world applications

Table 1. BENCHMARK SUITE. EVERY BENCHMARK IS FOLLOWED BY ITS VERSION NUMBER.

| | | | | |
|---|---|---|---|---|
| rm v9.7 | mkdir v9.7 | gzip v1.14 | make v4.4.1 | Nginx v1.28.0 |
| date v9.7 | sort v9.7 | grep v3.12 | lighttpd v1.4.79 | Redis v8.2.0 |
| uniq v9.7 | tar v1.35 | objdump v2.44 | wget v1.25.0 | SQLite v3.50.4 |
| chown v9.7 | bzip2 v1.0.8 | bftpd v6.3 | memcached v1.6.38 | |

- 16 libraries were instrumented as well

  Including complex ones like *glibc, OpenSSL*

# Cross-DSO Gadget Reduction

Table 2. NUMBER OF INDIRECTLY ACCESSIBLE CROSS-DSO ENBR64 PADS UNDER CET (COL. 3) AND PLATYPUS (COL. 4-6), FROM THE PERSPECTIVE OF THE RESPECTIVE MODULE. CT = CALLBACK TABLE.

| | Module | CET | PLATYPUS | CT | Red. (%) |
|---|---|---|---|---|---|
| Redis | redis-server | 3739 | 6 | 0 | 99.84 |
| | libc.so | 4961 | 52 | 52 | 98.95 |
| SQLite | sqlite3 | 6873 | 43 | 0 | 99.37 |
| | libreadline.so | 7452 | 3 | 2 | 99.96 |
| | libtinfo.so | 7856 | 3 | 0 | 99.96 |
| | libncurses.so | 7347 | 3 | 0 | 99.96 |
| | libm.so | 7242 | 18 | 18 | 99.75 |
| | libz.so | 8064 | 0 | 0 | 100.00 |
| | libc.so | 4472 | 40 | 40 | 99.11 |
| Nginx | nginx | 17986 | 6 | 0 | 99.97 |
| | libpcre2-8.so | 20124 | 2 | 2 | 99.99 |
| | libcrypto.so | 9551 | 84 | 83 | 99.12 |
| | libz.so | 20085 | 6 | 6 | 99.97 |
| | libssl.so | 17182 | 31 | 14 | 99.82 |
| | libcrypt.so | 20171 | 21 | 0 | 99.90 |
| | libc.so | 16533 | 166 | 166 | 98.99 |

# Cross-DSO Gadget Reduction

Table 2. NUMBER OF INDIRECTLY ACCESSIBLE CROSS-DSO ENBR64 PADS UNDER CET (COL. 3) AND PLATYPUS (COL. 4-6), FROM THE PERSPECTIVE OF THE RESPECTIVE MODULE. CT = CALLBACK TABLE.

| | Module | CET | PLATYPUS | CT | Red. (%) |
|---|---|---|---|---|---|
| Redis | redis-server | 3739 | 6 | 0 | 99.84 |
| | libc.so | 4961 | 52 | 52 | 98.95 |
| SQLite | sqlite3 | 6873 | 43 | 0 | 99.37 |
| | libreadline.so | 7452 | 3 | 2 | 99.96 |
| | libtinfo.so | 7856 | 3 | 0 | 99.96 |
| | libncurses.so | 7347 | 3 | 0 | 99.96 |
| | libm.so | 7242 | 18 | 18 | 99.75 |
| | libz.so | 8064 | 0 | 0 | 100.00 |
| | libc.so | 4472 | 40 | 40 | 99.11 |
| Nginx | nginx | 17986 | 6 | 0 | 99.97 |
| | libpcre2-8.so | 20124 | 2 | 2 | 99.99 |
| | libcrypto.so | 9551 | 84 | 83 | 99.12 |
| | libz.so | 20085 | 6 | 6 | 99.97 |
| | libssl.so | 17182 | 31 | 14 | 99.82 |
| | libcrypt.so | 20171 | 21 | 0 | 99.90 |
| | libc.so | 16533 | 166 | 166 | 98.99 |

# Cross-DSO Gadget Reduction

Table 2. NUMBER OF INDIRECTLY ACCESSIBLE CROSS-DSO ENBR64 PADS UNDER CET (COL. 3) AND PLATYPUS (COL. 4-6), FROM THE PERSPECTIVE OF THE RESPECTIVE MODULE. CT = CALLBACK TABLE.

| | Module | CET | PLATYPUS | CT | Red. (%) |
|---|---|---|---|---|---|
| Redis | redis-server | 3739 | 6 | 0 | 99.84 |
| | libc.so | 4961 | 52 | 52 | 98.95 |
| SQLite | sqlite3 | 6873 | 43 | 0 | 99.37 |
| | libreadline.so | 7452 | 3 | 2 | 99.96 |
| | libtinfo.so | 7856 | 3 | 0 | 99.96 |
| | libncurses.so | 7347 | 3 | 0 | 99.96 |
| | libm.so | 7242 | 18 | 18 | 99.75 |
| | libz.so | 8064 | 0 | 0 | 100.00 |
| | libc.so | 4472 | 40 | 40 | 99.11 |
| Nginx | nginx | 17986 | 6 | 0 | 99.97 |
| | libpcre2-8.so | 20124 | 2 | 2 | 99.99 |
| | libcrypto.so | 9551 | 84 | 83 | 99.12 |
| | libz.so | 20085 | 6 | 6 | 99.97 |
| | libssl.so | 17182 | 31 | 14 | 99.82 |
| | libcrypt.so | 20171 | 21 | 0 | 99.90 |
| | libc.so | 16533 | 166 | 166 | 98.99 |

# Cross-DSO Gadget Reduction

Table 2. NUMBER OF INDIRECTLY ACCESSIBLE CROSS-DSO ENBR64 PADS UNDER CET (COL. 3) AND PLATYPUS (COL. 4-6), FROM THE PERSPECTIVE OF THE RESPECTIVE MODULE. CT = CALLBACK TABLE.

| | Module | CET | PLATYPUS | CT | Red. (%) |
|---|---|---|---|---|---|
| Redis | redis-server | 3739 | 6 | 0 | 99.84 |
| | libc.so | 4961 | 52 | 52 | 98.95 |
| SQLite | sqlite3 | 6873 | 43 | 0 | 99.37 |
| | libreadline.so | 7452 | 3 | 2 | 99.96 |
| | libtinfo.so | 7856 | 3 | 0 | 99.96 |
| | libncurses.so | 7347 | 3 | 0 | 99.96 |
| | libm.so | 7242 | 18 | 18 | 99.75 |
| | libz.so | 8064 | 0 | 0 | 100.00 |
| | libc.so | 4472 | 40 | 40 | 99.11 |
| Nginx | nginx | 17986 | 6 | 0 | 99.97 |
| | libpcre2-8.so | 20124 | 2 | 2 | 99.99 |
| | libcrypto.so | 9551 | 84 | 83 | 99.12 |
| | libz.so | 20085 | 6 | 6 | 99.97 |
| | libssl.so | 17182 | 31 | 14 | 99.82 |
| | libcrypt.so | 20171 | 21 | 0 | 99.90 |
| | libc.so | 16533 | 166 | 166 | 98.99 |

# Cross-DSO Gadget Reduction

Table 2. NUMBER OF INDIRECTLY ACCESSIBLE CROSS-DSO ENBR64 PADS UNDER CET (COL. 3) AND PLATYPUS (COL. 4-6), FROM THE PERSPECTIVE OF THE RESPECTIVE MODULE. CT = CALLBACK TABLE.

| | Module | CET | PLATYPUS | CT | Red. (%) |
|---|---|---|---|---|---|
| Redis | redis-server | 3739 | 6 | 0 | 99.84 |
| | libc.so | 4961 | 52 | 52 | 98.95 |
| SQLite | sqlite3 | 6873 | 43 | 0 | 99.37 |
| | libreadline.so | 7452 | 3 | 2 | 99.96 |
| | libtinfo.so | 7856 | 3 | 0 | 99.96 |
| | libncurses.so | 7347 | 3 | 0 | 99.96 |
| | libm.so | 7242 | 18 | 18 | 99.75 |
| | libz.so | 8064 | 0 | 0 | 100.00 |
| | libc.so | 4472 | 40 | 40 | 99.11 |
| Nginx | nginx | 17986 | 6 | 0 | 99.97 |
| | libpcre2-8.so | 20124 | 2 | 2 | 99.99 |
| | libcrypto.so | 9551 | 84 | 83 | 99.12 |
| | libz.so | 20085 | 6 | 6 | 99.97 |
| | libssl.so | 17182 | 31 | 14 | 99.82 |
| | libcrypt.so | 20171 | 21 | 0 | 99.90 |
| | libc.so | 16533 | 166 | 166 | 98.99 |

> 98% gadget reduction per module

# FOP Attacks Mitigation

- *_dl_call_fini*: part of glibc loader
- Called gadgets: part of libc

```
----|  11. Appendix B: Intel "/bin/sh" in memory chain

+--------------------------+--------------------------+
|       Function Name      |   Equivalent Operation   |
+--------------------------+--------------------------+
| _nss_files_endpwent      | MOV RDI, 0x6             |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| __cache_sysconf          | SUB RDI, 0xB9            |
| _dl_mcount_wrapper...    | MOV RSI, RDI             |
```

```
void
_dl_call_fini (void *closure_map)
{
  struct link_map *map = closure_map;

  /* Make sure nothing happens if we are called twice.  */
  map->l_init_called = 0;

  ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
  if (fini_array != NULL)
    {
      ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                          + fini_array->d_un.d_ptr);
      size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                   / sizeof (ElfW(Addr)));

      while (sz-- > 0)
        ((fini_t) array[sz]) ();
    }

  /* Next try the old-style destructor.  */
  ElfW(Dyn) *fini = map->l_info[DT_FINI];
  if (fini != NULL)
    DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

dispatcher loop

# FOP Attacks Mitigation

- *_dl_call_fini*: part of glibc loader
- Called gadgets: part of libc

No PLTs for them inside the loader

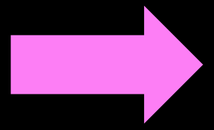
```
void
_dl_call_fini (void *closure_map)
{
  struct link_map *map = closure_map;

  /* Make sure nothing happens if we are called twice.  */
  map->l_init_called = 0;

  ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
  if (fini_array != NULL)
    {
      ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                          + fini_array->d_un.d_ptr);
      size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                   / sizeof (ElfW(Addr)));

      while (sz-- > 0)
        ((fini_t) array[sz]) ();
    }

  /* Next try the old-style destructor.  */
  ElfW(Dyn) *fini = map->l_info[DT_FINI];
  if (fini != NULL)
    DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

dispatcher loop

# FOP Attacks Mitigation

- *_dl_call_fini*: part of glibc loader
- Called gadgets: part of libc

No PLTs for them inside the loader



Transitions impossible

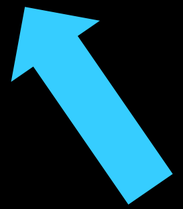
```
void
_dl_call_fini (void *closure_map)
{
  struct link_map *map = closure_map;

  /* Make sure nothing happens if we are called twice.  */
  map->l_init_called = 0;

  ElfW(Dyn) *fini_array = map->l_info[DT_FINI_ARRAY];
  if (fini_array != NULL)
    {
      ElfW(Addr) *array = (ElfW(Addr) *) (map->l_addr
                                          + fini_array->d_un.d_ptr);
      size_t sz = (map->l_info[DT_FINI_ARRAYSZ]->d_un.d_val
                   / sizeof (ElfW(Addr)));

      while (sz-- > 0)
        ((fini_t) array[sz]) ();
    }

  /* Next try the old-style destructor.  */
  ElfW(Dyn) *fini = map->l_info[DT_FINI];
  if (fini != NULL)
    DL_CALL_DT_FINI (map, ((void *) map->l_addr + fini->d_un.d_ptr));
}
```

dispatcher loop

# FOP Attacks Mitigation

- Rare and hard to find
- Most of them not exploitable
- Exploitable ones are related to initialization and finalization routines

LOOP attack: *_initterm()* in *msvcrt.dll*
Phrack attack: *_dl_call_fini()* in *ld-linux*

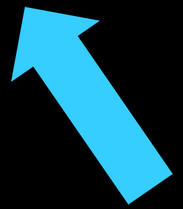


We will revisit this later

# FOP Attacks Mitigation

- Rare and hard to find
- Most of them not exploitable
- Exploitable ones are related to initialization and finalization routines

LOOP attack: *_initterm()* in *msvcrt.dll*
Phrack attack: *_dl_call_fini()* in *ld-linux*



Dedicated callback tables at call sites

# Intra-DSO Protection

- Normal PLTs are not reachable indirectly

# Intra-DSO Protection

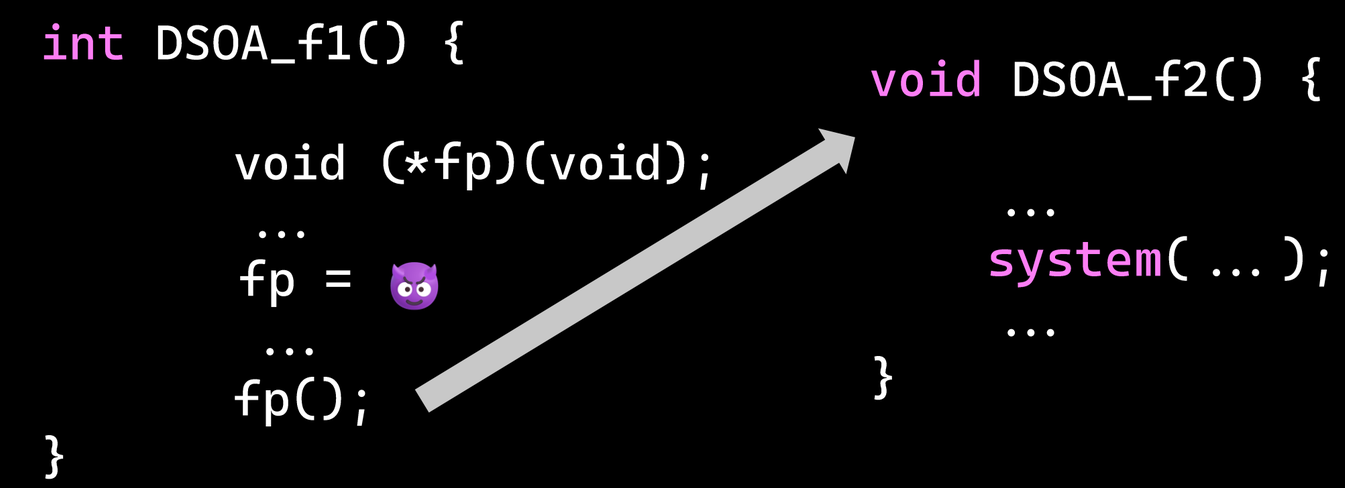
- Normal PLTs are not reachable indirectly

```
int DSOA_f1() {

        void (*fp)(void);
        ...
        fp = 😈
        ...
        fp();
}
```

```
void DSOA_f2() {

        ...
        system( ... );
        ...
}
```

# Intra-DSO Protection

- Normal PLTs are not reachable indirectly

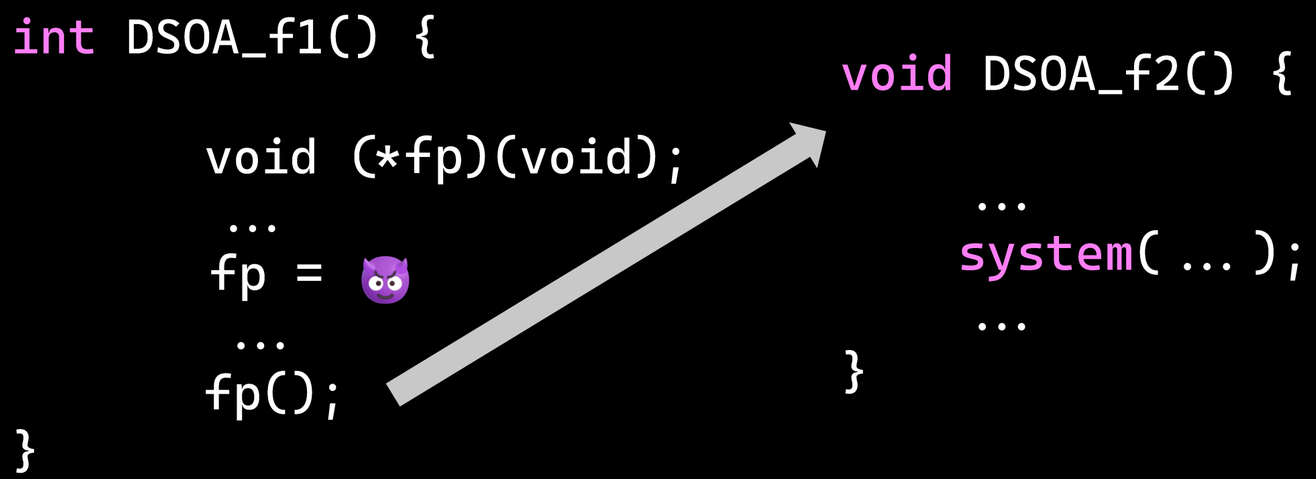


# Intra-DSO Protection

- Normal PLTs are not reachable indirectly

- Isolate *DSOA_f2* outside the EJ of DSO A

  If *DSOA_f2* is never called indirectly (static analysis)

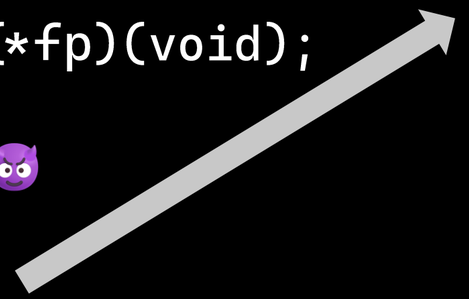


# Intra-DSO Protection

- Normal PLTs are not reachable indirectly

```
int DSOA_f1() {
        void (*fp)(void);
        ...
        fp = 😈
        ...
        fp();
}
```

```
void DSOA_f2() {
        ...
        system( ... );
        ...
}
```

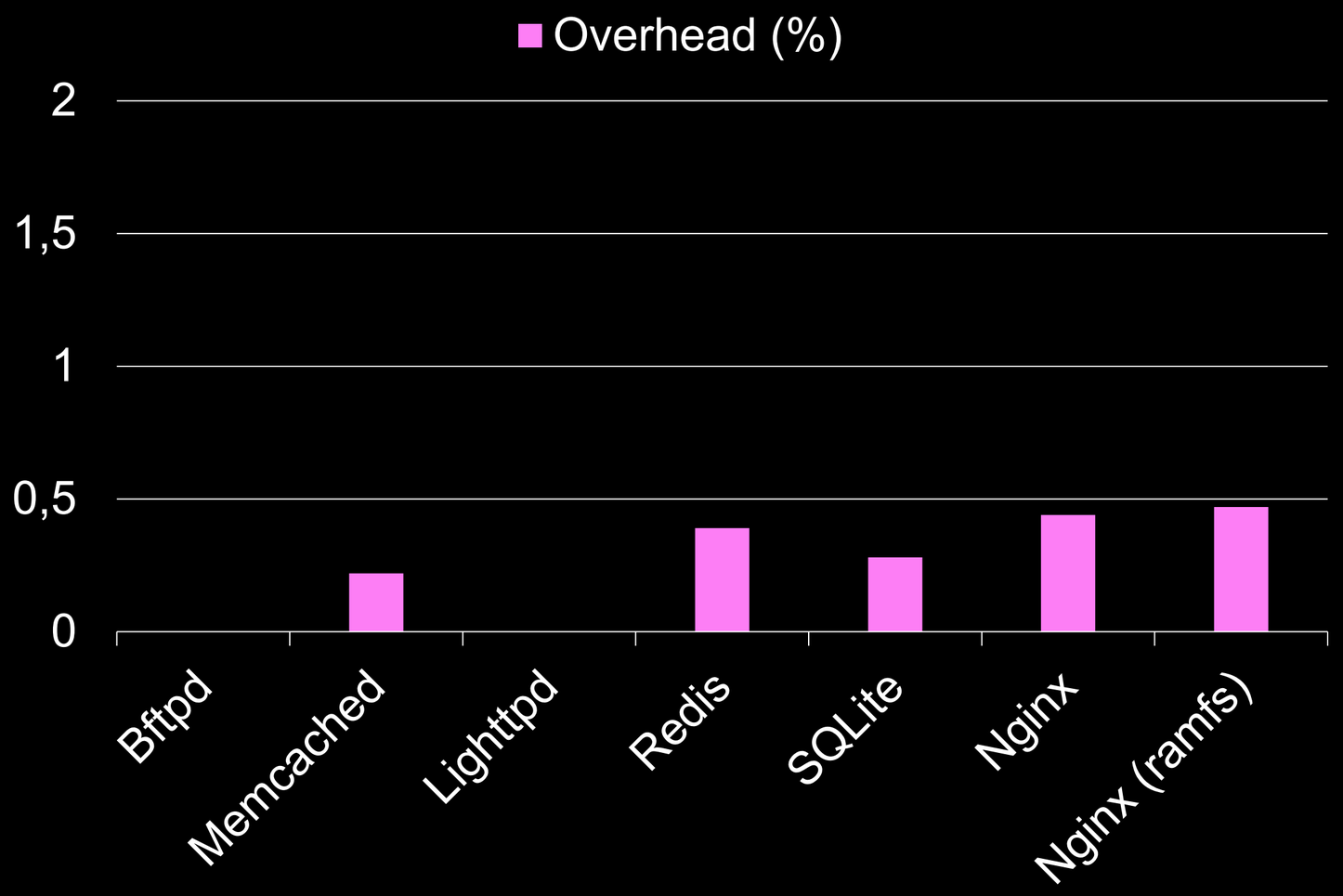


- Isolate *DSOA_f2* outside the EJ of DSO A

  If *DSOA_f2* is never called indirectly (static analysis)

Works for sensitive functions such as *system()* in libc

# Performance



# DISCUSSION

# Modular Support

Interoperability with unprotected modules is supported.

- Unprotected modules can reach arbitrary functions within the address space

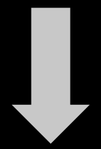
- Protected modules can only reach unprotected functions for which they possess PLT stubs

# Modular Support

Interoperability with unprotected modules is supported.

- Unprotected modules can reach arbitrary functions within the address space

- Protected modules can only reach unprotected functions for which they possess PLT stubs



Reachability of 🐞 in third-party code is limited

# General Applicability

## ARM

- ARM BTI ≈ Intel IBT
- Methodology and design remain the same
- Need for backward-edge protection

  No shadow stack… use PAC

# General Applicability

## ARM

- ARM BTI ≈ Intel IBT
- Methodology and design remain the same
- Need for backward-edge protection

  No shadow stack… use PAC

## Microsoft Windows

- CFG ≈ Intel IBT
- Leverage IAT instead of PLTs
- Possible modifications to OS-specific code paths

  E.g., loader, system libraries

# PLaTypus: Conclusion

- Closes the gap of arbitrary cross-DSO transitions
- Mitigates state-of-the-art FOP attacks
- Supports interoperability between protected and unprotected code
- Incurs negligible overhead while handling complex corner cases

# Interested in more?

- IEEE S&P paper here
- Code here


## PLATYPUS: Restricting Cross-Module Transitions to Mitigate Code-Reuse Attacks

Apostolos Chatzianagnostou
*CISPA Helmholtz Center for Information Security*

Marcos Bajo
*CISPA Helmholtz Center for Information Security*

Christian Rossow
*CISPA Helmholtz Center for Information Security*

***Abstract*—Numerous techniques have been proposed to thwart code reuse attacks, yet practical adoption remains limited due to compatibility and deployment challenges. In the current and foreseeable Intel architecture landscape, the main line of defense against such attacks is Intel CET—a hardware-enforced control-flow integrity mechanism integrated into recent Intel x86-64 CPUs. However, despite its hardware-backed protections and widespread adoption, CET still provides only partial security: it continues to allow hijacked function pointers to invoke arbitrary functions *across* module boundaries, a capability that remains fundamental to many modern exploits.**

**This paper proposes PLATYPUS, a novel defense on top of Intel CET to address this limitation. PLATYPUS enforces *execution jails* using lightweight address masking to ensure indirect control transfers remain within module boundaries. Cross-DSO function calls are only permitted via necessary PLT stubs specific to each DSO. The evaluation on our LLVM-based prototype, spanning 19 applications and 16 shared libraries (including glibc), demonstrates that PLATYPUS reduces indirectly accessible cross-DSO functions by over 98%. Performance testing with complex applications like Nginx and Redis shows that PLATYPUS incurs no more than 0.5% overhead.**

legitimate, precomputed transfers at runtime. Numerous CFI schemes have been proposed in recent years. These schemes are typically categorized based on the strictness of their CFGs as either coarse-grained or fine-grained, with the latter raising the bar for successful exploitation considerably. Despite extensive academic efforts to develop fine-grained CFI techniques, their adoption in practice remains notably limited [15]. A core reason for this is their lack of robust support for interoperability between protected and unprotected code [10], [46], a critical requirement in real-world environments where instrumented applications must frequently interact with third-party libraries.

Therefore, the vast majority of modern software relies on coarse-grained CFI schemes as the primary line of defense. The prevailing solutions, whose adoption is steadily increasing and is expected to become the default for future systems on the two most popular architectures are Intel's Control-Flow Enforcement Technology (CET) [40] and Arm's Branch Target Identification (BTI) [11], respectively. Both restrict the modularity of code-reuse gadgets to entire functions. This restriction is significant, as it complicates an attacker's ability to set up the necessary function arguments despite successfully hijacking control flow. Nonetheless,